

“AUTOMATED NETWORK INTRUSION DETECTION FOR INTERNET OF THINGS: SECURITY ENHANCEMENTS”

A project stage -II report submitted to jntuh in the partial fulfillment of the requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING**

Submitted By

Batch No. – 02

A. VAISHNAVI	- 21631A0559
D. ABHIRAM	- 21631A0502
M. SAISHASHI	- 21631A0543
T. SAIKUMAR	- 21631A0542

Under the Esteemed guidance of

Dr. T. Madhu, M. Tech, PhD

Associate Professor in CSE Dept.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



SRI VENKATESWARA ENGINEERING COLLEGE

(SPONSORED BY THE EXHIBITION SOCIETY, HYDERABAD)

(Affiliated to Jawaharlal Nehru Technological University, Hyderabad)

(Approved by AICTE, NEW DELHI)

SURYAPET – 508213

JUNE-2025

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SRI VENKATESWARA ENGINEERING COLLEGE

SURYAPET-508213



CERTIFICATE

This is to certify that the project report entitled “**AUTOMATED NETWORK INTRUSION DETECTION FOR INTERNET OF THINGS: SECURITY ENHANCEMENTS**” is the bonafide work done by **A. VAISHNAVI (21631A0559), D. ABHIRAM (21631A0502), M.SAI SHASHI (21631A0543), T.SAI KUMAR (21631A0542)**, in partial fulfillment of the requirements for the award of **BACHELOR OF TECHNOLOGY** in **Computer Science and Engineering** by JNTUH during the academic year 2024-25.

Guide

Dr.T. MADHU, M Tech, PhD
Associate Professor,
Department of C.S.E.,
Sri Venkateswara Engineering College,
Suryapet.

Head of the Department

Mr. P. RATHAIAH, M Tech (CSE)
Sr. Assistant Professor,
Department of C.S.E.,
Sri Venkateswara Engineering College,
Suryapet.

Principal

Dr. Raju Muthyala, M Tech, PhD
Sri Venkateswara Engineering College, Suryapet.

External Examiner

DECLARATION

We, A. Vaishnavi, D. Abhiram, M. Sai Shashi, T. Saikumar, students of B. Tech with CSE of Sri Venkateswara Engineering College, Suryapet, being H.T. No 21631A0559, 21631A0502, 21631A0543, 21631A0542 respectively at this moment declare that the project with title “AUTOMATED NETWORK INTRUSION DETECTION FOR INTERNET OF THINGS: SECURITY ENHANCEMENTS” is the original work done by us.

To the best of our knowledge and belief, we at this moment declare that this project bears no resemblance to any other project submitted at Sri Venkateswara Engineering College, Suryapet, or any other college affiliated with Jawaharlal Nehru Technological University, Hyderabad for the award of the degree.

Place: Suryapet

Date:

Signature of the candidate

1. A. Vaishnavi
2. D. Abhiram
3. M. Sai Shashi
4. T. Saikumar

ACKNOWLEDGEMENT

We thank the almighty for giving us the courage and perseverance to complete the project. This project itself is an acknowledgment for all those people who have given this project a success.

We take this opportunity to express our deep and sincere gratitude to the project guide **Dr. T. MADHU**, MTech, Associate Professor for his valuable advice at every stage of this work. Without his supervision and valuable guidance, we would never have come out in this form.

We are also thankful to **MR. P. RATHAIAH** M. Tech, Sr. Assistant Professor Head of the Department of Computer Science and Engineering, **Dr. RAJU MUTHYALA** MTech, PhD Principal S.V.E.S for providing excellent facilities, motivation, and encouragement to complete the project work on time.

Last but not least we would like to express our deep sense of gratitude and earnest thanksgiving to our parents for their moral support and heart felt cooperation in doing the project. We would also like to thank all the teaching and non-teaching staff and my friends whose direct or indirect help has enabled us to complete this work successfully.

PROJECT ASSOCIATE

A. Vaishnavi – 21631A0559

D. Abhiram -21631A0502

M. Saishashi – 21631A0543

T. Saikumar – 21631A0543

ABSTRACT

As interconnected devices increasingly transmit personal and sensitive data, security attacks are becoming more sophisticated and prevalent, highlighting the critical need for effective security solutions in Internet of Things (IoT) environments. An automated Network Intrusion Detection (NID) system plays a vital role in notifying system administrators of security breaches, acting as an efficient tool for protecting IoT networks from various threats. This study utilizes the UNSW-NB 15 dataset to enhance intrusion detection accuracy by addressing performance challenges and class imbalances within the data. We employ a combination of feature selection techniques, including Filter Method, Wrapper Method, and an Embedded approach using Lasso and Random Forest with Recursive Feature Elimination (RFE), alongside Pearson Correlation Coefficient (PCC). To tackle class imbalance, we apply the Synthetic Minority Over-sampling Technique (SOMTE). Various algorithms are implemented, including Random Forest, Decision Tree, AdaBoost, Bernoulli Naive Bayes, K-Nearest Neighbors, and Logistic Regression. Notably, the Stacking Classifier, which combines Boosted Decision Trees, Bagging with Random Forest, and LightGBM, demonstrates high performance in accurately detecting intrusions, significantly improving detection rates and reducing false alarms.

TABLE OF CONTENTS

S.NO	CONTENT	PGNO
CHAPTER:1	Introduction	1
	1.1 Objective	1
	1.2 Problem Statement	2
	1.3 Software requirements	3
	1.4 Hardware requirements	3
CHAPTER:2	Feasibility study	4
CHAPTER:3	Literature survey	5-6
CHAPTER:4	System analysis	
	4.1 Existing system	7
	4.1.1 Disadvantages of existing system	7
	4.2 Proposed system	8
	4.2.1 Advantages of proposed system	8
	4.3 Functional requirements	9
	4.4 Non-Functional requirements	9
CHAPTER:5	System design	
	5.1 System architecture	10
	5.2 UML diagrams	12
CHAPTER:6	Implementation	
	6.1 Modules	19
	6.2 Sample code	22
CHAPTER:7	Software environment	31
CHAPTER:8	System testing	39
	8.1 Testing strategies	39
	8.2 Test cases	40
CHAPTER:9	Screens	41
CHAPTER:10	Conclusion	60
CHAPTER:11	Future Scope	61
CHAPTER:12	References	62

LIST OF FIGURES

Following is the list of figures used in this project documentation at various locations

FIGURE NO.	DESCRIPTION	PGNO
5.1.1	System architecture	10
5.2.1	Use case diagram	13
5.2.2	Class diagram	14
5.2.3	Activity diagram	15
5.2.4	Sequence diagram	16
5.2.5	Collaboration diagram	17
5.2.6	Component diagram	18
5.2.7	Deployment diagram	18

Chapter 1: INTRODUCTION

The proliferation of internet use has markedly increased the quantity of sensitive and critical personal information sent over the World Wide Web. As a result, attackers have started using security weaknesses to get unauthorised access to critical information. Such compromises may jeopardise the availability and confidentiality of data. Cybersecurity has become a crucial domain due to its fundamental role in thwarting intrusions. The Network Intrusion Detection (NID) system is essential inside this security architecture. A Network Intrusion Detection (NID) system oversees computer systems and network traffic, analysing activities to detect intrusions. Besides identifying intrusions, overseeing network activity, and signalling suspicious or malicious conduct, it also highlights policy infringements. Monitoring contemporary dangers has become more accessible for network managers. Currently, Machine/Deep Learning (M/DL) techniques are acknowledged as efficient methods for processing extensive data, resulting in enhanced computational results. Optimal features are chosen from the dataset using different machine learning approaches, which are then used for classification. Alternatively, characteristics are automatically retrieved from the dataset via deep learning techniques according to their significance. In IoT networks, cybersecurity issues have become a critical concern, which may be mitigated by carefully designing and implementing a strong Intrusion Detection System (IDS) at the edge nodes. In recent decades, machine learning and deep learning-based network intrusion detection systems have shown significant efficacy in detecting assaults inside Internet of Things networks. As a result, their popularity and effectiveness in detecting dangers inside networks have increased in cybersecurity.

1.1 Objective:

The objective of this study is to develop an automated Network Intrusion Detection (NID) system specifically designed for Internet of Things (IoT) environments. This system aims to enhance the detection of security breaches by employing various machine learning algorithms and advanced feature selection techniques. By addressing class imbalance and optimizing detection accuracy, the research seeks to provide a robust solution that effectively safeguards IoT networks against a wide range of cyber threats while minimizing false alarms.

1.2 Problem Statement:

- The rapid growth of interconnected devices leads to increasingly sophisticated security attacks, making traditional security measures inadequate for the unique challenges posed by the dynamic nature of Internet of Things (IoT) environments.
- Current intrusion detection systems struggle with high false alarm rates and class imbalance in datasets, particularly when applied to IoT networks, resulting in a critical need for more accurate and efficient detection methodologies.
- System administrators and end-users of IoT devices are primarily affected by security breaches, facing potential data loss, privacy violations, and disruptions in services, which can undermine trust in IoT technology and its applications.
- Security breaches can lead to significant financial losses for organizations, regulatory penalties, and damage to reputation, fostering an environment of vulnerability that discourages the adoption of IoT solutions across various industries.
- To address these challenges, we will develop an automated Network Intrusion Detection system utilizing advanced machine learning algorithms and feature selection techniques to enhance detection accuracy and reduce false alarms in IoT environments.

1.3 SOFTWARE REQUIREMENTS

- 1) Software: Anaconda
- 2) Primary Language: Python
- 3) Frontend Framework: Flask
- 4) Back-end Framework: Jupyter Notebook
- 5) Database: Sqlite3
- 6) Front-End Technologies: HTML, CSS, JavaScript and Bootstrap4

1.4 HARDWARE REQUIREMENTS

- 1) Operating System: Windows
- 2) Processor: i5 and above
- 3) Ram: 8GB and above
- 4) Hard Disk: 25 GB in local drive

Chapter 2: FEASIBILITY STUDY

A feasibility study—sometimes called a feasibility analysis or feasibility report—is a way to evaluate whether or not a project plan could be successful.

Types of feasibility studies:

There are five main types of feasibility studies: technical feasibility, financial feasibility, market feasibility (or market fit), operational feasibility, and legal feasibility.

Technical feasibility:

A technical feasibility study reviews the technical resources available for your project. This study determines if you have the right equipment, enough equipment, and the right technical knowledge to complete your project objectives.

Financial feasibility:

Financial feasibility describes whether or not your project is fiscally viable. A financial feasibility report includes a cost-benefit analysis of the project..

Market feasibility:

The market feasibility study is an evaluation of how your team expects the project's deliverables to perform in the market.

Operational feasibility:

An operational feasibility study evaluates whether or not your organization is able to complete this project.

Legal feasibility:

A legal feasibility analysis assesses whether the proposed project complies with all relevant legal requirements and regulations. This includes examining legal and regulatory barriers, necessary permits, licenses, or certifications, potential legal liabilities or risk.

Chapter 3: LITERATURE SURVEY

3.1 An improved PIO feature selection algorithm for IoT network intrusion detection system based on ensemble learning:

ABSTRACT: With the rapid growth of the number of connected devices that exchange personal, sensitive, and important data through the IoT based global network, attacks that are targeting security services are increasing as well. Accordingly, there is a need for security solutions that are suitable for IoT environment. A network intrusion detection system (NIDS) is a solution that examines network traffic and alerts system administrators if there are security breaches. In this paper, an enhanced version of pigeon-inspired optimization (PIO) is proposed which enhances PIO by adding a local search algorithm named (LS-PIO). Moreover, an ensemble learning approach, which is based on multiple one-class classifiers, has been used in order to improve the performance of proposed NIDS. Four benchmark datasets were used to evaluate the LS-PIO and ensemble based NIDS which are BoT-IoT, UNSW-NB15, NLS-KDD and KDDCUP99. The evaluation took into consideration F-score, accuracy, AUC, FPR and TPR. Results show that the suggested approach outperforms other NIDS techniques that are selected from state-of-the-art relevant research found in the literature.

3.2 Anomaly based network intrusion detection for IoT attacks using deep learning technique:

ABSTRACT: Internet of Things (IoT) applications are growing in popularity for being widely used in many real-world services. In an IoT ecosystem, many devices are connected with each other via internet, making IoT networks more vulnerable to various types of cyber attacks, thus a major concern in its deployment is network security and user privacy. To protect IoT networks against various attacks, an efficient and practical Intrusion Detection System (IDS) could be an effective solution. In this paper, a novel anomaly-based IDS system for IoT networks is proposed using Deep Learning technique. Particularly, a filter-based feature selection Deep Neural Network (DNN) model where highly correlated features are dropped has been presented. Further, the model is tuned with various parameters and hyper parameters. The UNSW-NB15 dataset comprising of four attack classes is utilized for this purpose.

3.3 TAD: Transfer learning-based multi-adversarial detection of evasion attacks against network intrusion detection systems:

ABSTRACT: Nowadays, intrusion detection systems based on deep learning deliver state-of-the-art performance. However, recent research has shown that specially crafted perturbations, called adversarial examples, are capable of significantly reducing the performance of these intrusion detection systems. The objective of this paper is to design an efficient transfer learning-based adversarial detector and then to assess the effectiveness of using multiple strategically placed adversarial detectors compared to a single adversarial detector for intrusion detection systems. In our experiments, we implement existing state-of-the-art models for intrusion detection. We then attack those models with a set of chosen evasion attacks. In an attempt to detect those adversarial attacks, we design and implement multiple transfer learning-based adversarial detectors, each receiving a subset of the information passed through the IDS. By combining their respective decisions, we illustrate that combining multiple detectors can further improve the detectability of adversarial traffic compared to a single detector in the case of a parallel IDS design.

3.4 Robust unsupervised network intrusion detection with self-supervised masked context reconstruction:

ABSTRACT: Modern network intrusion detection systems often use deep learning to enhance their intelligence and feature learning. However, due to limited labeled data and the need for early warnings, many systems rely on unsupervised anomaly detection. These methods often overlook the importance of time-related patterns and the presence of anomalies in training data, which can reduce accuracy. To address these issues, we propose RUIDS (Robust Unsupervised Intrusion Detection System). It uses a transformer-based self-supervised learning approach combined with a masked context reconstruction module. This setup helps the model learn temporal relationships and build stronger, less noise-sensitive representations. We tested RUIDS on four datasets, showing strong performance and robustness. Notably, it outperformed the second-best method by 9.04% on UNSW-NB15 and 9.58% on CICIDS-WED in AUC scores. Even when we increased the amount of anomaly contamination, the system maintained high performance. An ablation study further confirmed the value of both key components: the self-supervised learning approach and the reconstruction module.

Chapter 4: SYSTEM ANALYSIS

4.1 EXISTING SYSTEM:

Existing systems for network intrusion detection have explored various advanced algorithms to enhance classification performance. One approach utilizes a Recurrent Neural Network (RNN) for binary classification, evaluated on the UNSW-NB15 dataset, demonstrating high accuracy. Another model employs the Gated Recurrent Unit (GRU) framework and direct flow inspection to protect Software-Defined Networks (SDN) against intrusions, assessed using the CICDDoS2019 and CSE-CIC-IDS2018 datasets. This method has achieved notable recall, accuracy, and F1-score metrics. Additionally, improvements in anomaly detection have been achieved through the use of Long Short-Term Memory (LSTM) networks combined with the Attention Mechanism. This approach benefits from the SMOTE algorithm to address class imbalance, resulting in high accuracy. Comparative studies on RNN, LSTM, and GRU models, using datasets like CSE-CIC-IDS2018 and CIC-IDS2017, reveal that these models offer comparable performance in terms of accuracy.

4.1.1 DISADVANTAGES OF EXISTING SYSTEM:

1. Existing systems primarily focus on a limited range of neural network architectures, which may not leverage the full potential of ensemble methods and advanced feature selection techniques, potentially leading to suboptimal detection performance in diverse scenarios.
2. The reliance on individual models like RNN, GRU, and LSTM without integrating their strengths through ensemble techniques may result in a lack of robustness, limiting their adaptability to evolving intrusion patterns in complex network environments.
3. Existing methods often address class imbalance using traditional techniques, which may not be sufficient for real-time detection, leading to potential biases and misclassification, especially in IoT environments where data distribution can vary significantly.
4. Many current systems have not fully optimized the feature extraction process, which can affect the quality of data fed into the models, potentially hindering overall accuracy and the effectiveness of the detection mechanisms.

4.2 Proposed System:

In this project, we propose an advanced automated Network Intrusion Detection (NID) system tailored for Internet of Things (IoT) environments. The system will employ various feature selection techniques, including Filter Method, Wrapper Method, and an Embedded approach combining Lasso with Random Forest and Recursive Feature Elimination (RFE) to enhance data quality and relevance. To address class imbalance, the Synthetic Minority Over-sampling Technique (SOMTE) will be utilized, ensuring a more balanced dataset for training. We will implement multiple machine learning algorithms, including Random Forest, Decision Tree, AdaBoost, Bernoulli Naive Bayes, K-Nearest Neighbors, and Logistic Regression, to improve intrusion detection accuracy. The system will also feature a Stacking Classifier that combines the predictions of Boosted Decision Trees, Bagging with Random Forest, and LightGBM, thereby leveraging their strengths for a robust detection mechanism. This integrated approach aims to create a more reliable NID system capable of effectively safeguarding IoT networks against various security threats.

4.2.1 Advantages of proposed system:

1. The proposed system incorporates multiple feature selection techniques, enhancing the quality and relevance of input data, which improves model performance and facilitates better learning, ultimately leading to more accurate intrusion detection in IoT environments.
2. By utilizing the Synthetic Minority Over-sampling Technique (SOMTE) to address class imbalance, the proposed system ensures a more balanced dataset, reducing the risk of bias and improving the overall reliability of the intrusion detection process.
3. The integration of various machine learning algorithms allows the proposed system to leverage different strengths, thereby enhancing detection capabilities and providing a comprehensive approach that can adapt to diverse security threats in IoT networks.
4. The implementation of a Stacking Classifier combines predictions from multiple algorithms, increasing the robustness and accuracy of intrusion detection, as it harnesses the benefits of ensemble learning to effectively counteract a wide range of security threats.

4.3 FUNCTIONAL REQUIREMENTS

1. Data Collection
2. Data Pre-processing
3. Training and Testing
4. Modelling
5. Predicting

4.4 NON FUNCTIONAL REQUIREMENTS

Performance

The system should efficiently process incoming data from IoT devices, providing real-time intrusion detection with minimal latency to ensure timely responses to security threats without compromising overall network performance.

Scalability

The NID system must be designed to accommodate an increasing number of IoT devices, maintaining performance levels as the network expands and adapting seamlessly to varying data volumes and user demands.

Reliability

The system should consistently operate without failures, ensuring continuous monitoring and detection capabilities for IoT networks.

Usability

The user interface of the NID system should be intuitive and user-friendly, allowing system administrators to easily navigate, configure settings

Security

The system must implement stringent security measures to protect against unauthorized access and data breaches. This includes secure data transmission protocols, regular updates to security features, and thorough access control mechanisms for user authentication.

CHAPTER 5: SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE:

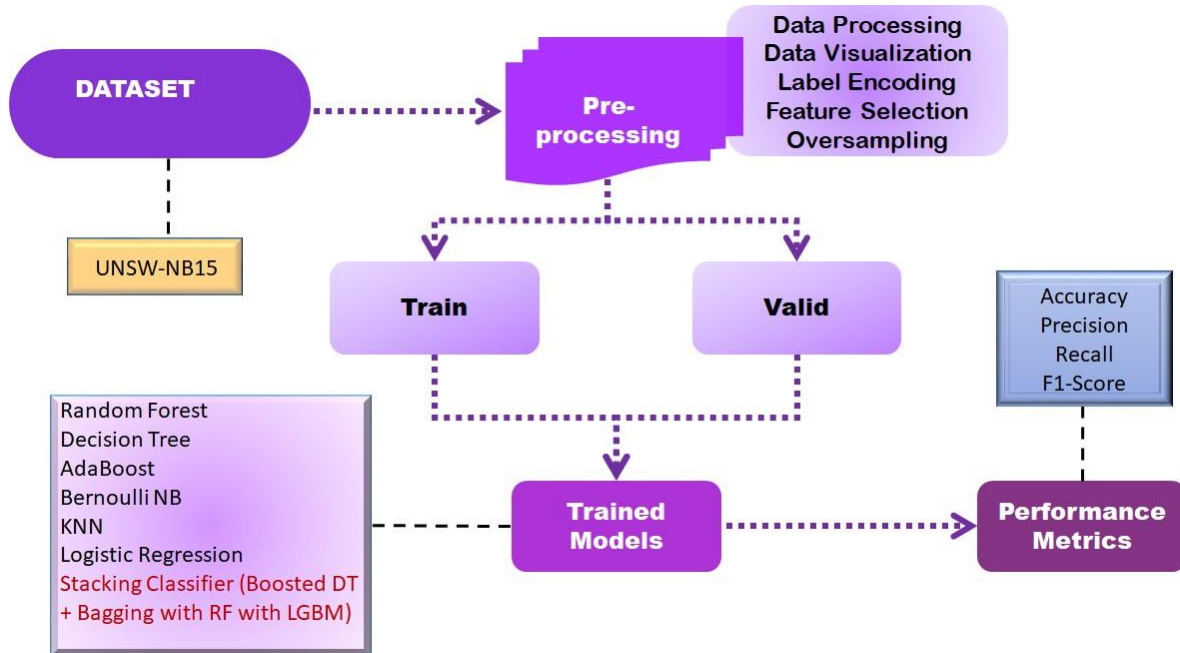
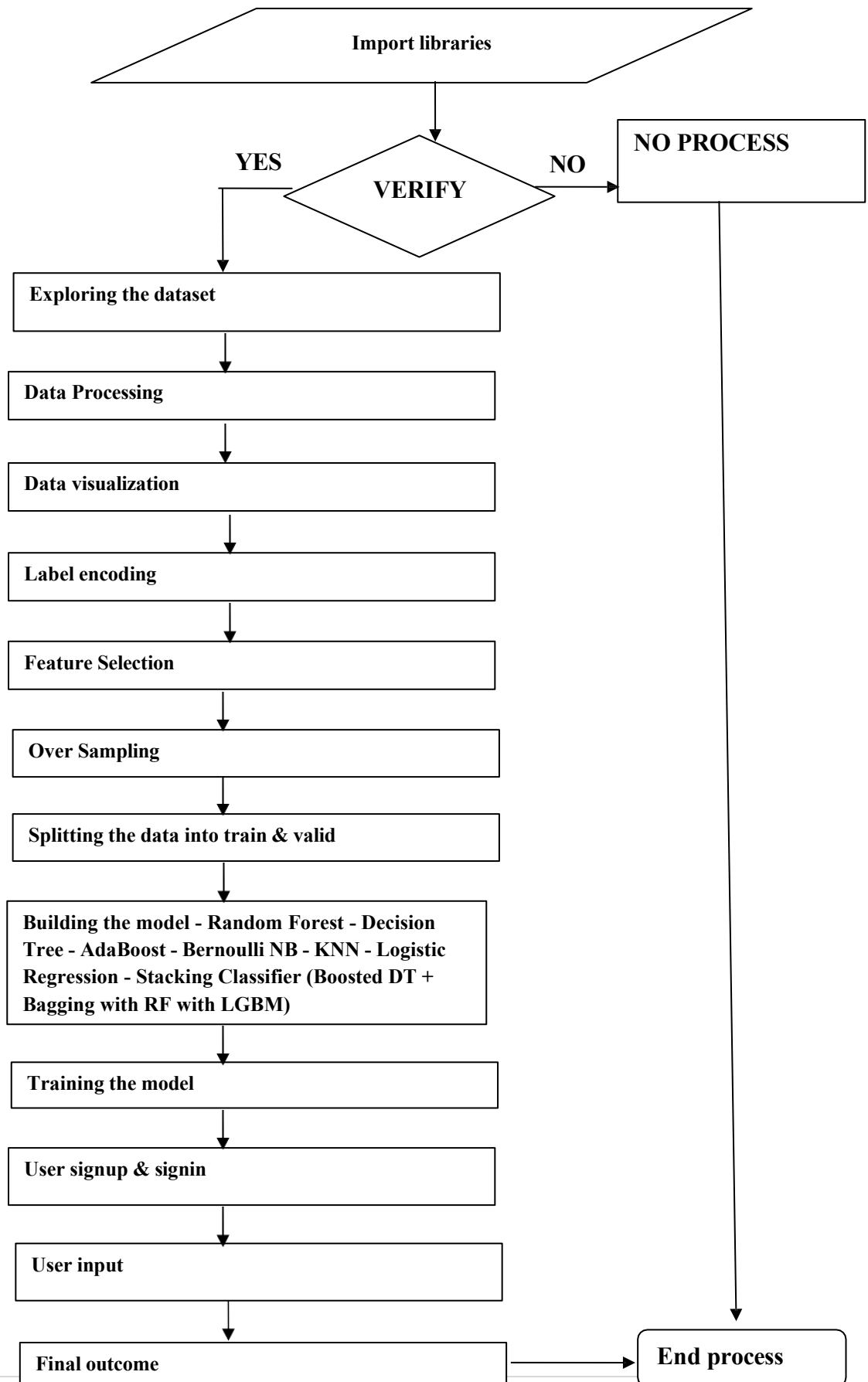


Fig.5.1.1 System architecture

DATA FLOW DIAGRAM:

A Data Flow Diagram (DFD) is a visual representation that illustrates how data moves through a system, highlighting the processes involved, data stores, and the interactions between various components. In the context of an automated Network Intrusion Detection (NID) system for Internet of Things (IoT) environments, a DFD can effectively map out the flow of data from IoT devices to the NID system, showing how incoming data is processed for intrusion detection. It delineates the inputs, outputs, and storage of information while clarifying the relationships between different processes, such as data collection, analysis, and alert generation. This graphical representation aids in understanding the overall system architecture and helps identify areas for optimization and improvement in data handling.



5.2 UML DIAGRAMS

Unified Modeling Language (UML) is a standardized visual language used to create models of software systems, providing a way to represent various aspects of system architecture and design. In the context of an automated Network Intrusion Detection (NID) system for Internet of Things (IoT) environments, UML can be utilized to depict the system's structure, behavior, and interactions. UML diagrams, such as use case diagrams, class diagrams, and sequence diagrams, help illustrate how different components of the NID system work together to detect and respond to security threats. By providing a clear visual representation of the system's components and their relationships, UML facilitates effective communication of design ideas and aids in ensuring that the system meets its functional and non-functional requirements.

Goals of UML

UML aims to provide a clear visual representation of the automated Network Intrusion Detection system, helping to illustrate the relationships and interactions between various components in the context of IoT environments.

It facilitates effective communication among project contributors, ensuring that all parties have a shared understanding of the system's design and functionality, which is essential for successful collaboration.

The use of UML diagrams assists in identifying and defining system requirements, allowing for a structured approach to capturing the necessary functionalities and behaviors of the NID system.

UML helps in documenting the system architecture, providing a comprehensive overview that can guide future development, maintenance, and enhancements of the NID system as IoT security needs evolve.

By utilizing UML, the project can effectively model complex behaviors and processes involved in intrusion detection, enabling better analysis and optimization of the system's performance in real-time scenarios.

Use Case Diagram

A use case diagram visually represents the interactions between actors and the system, outlining use cases like user signup, data processing, and model training. Actors include users and system components, while associations illustrate their relationships, facilitating clarity in functional requirements and user interactions for the automated NID system.

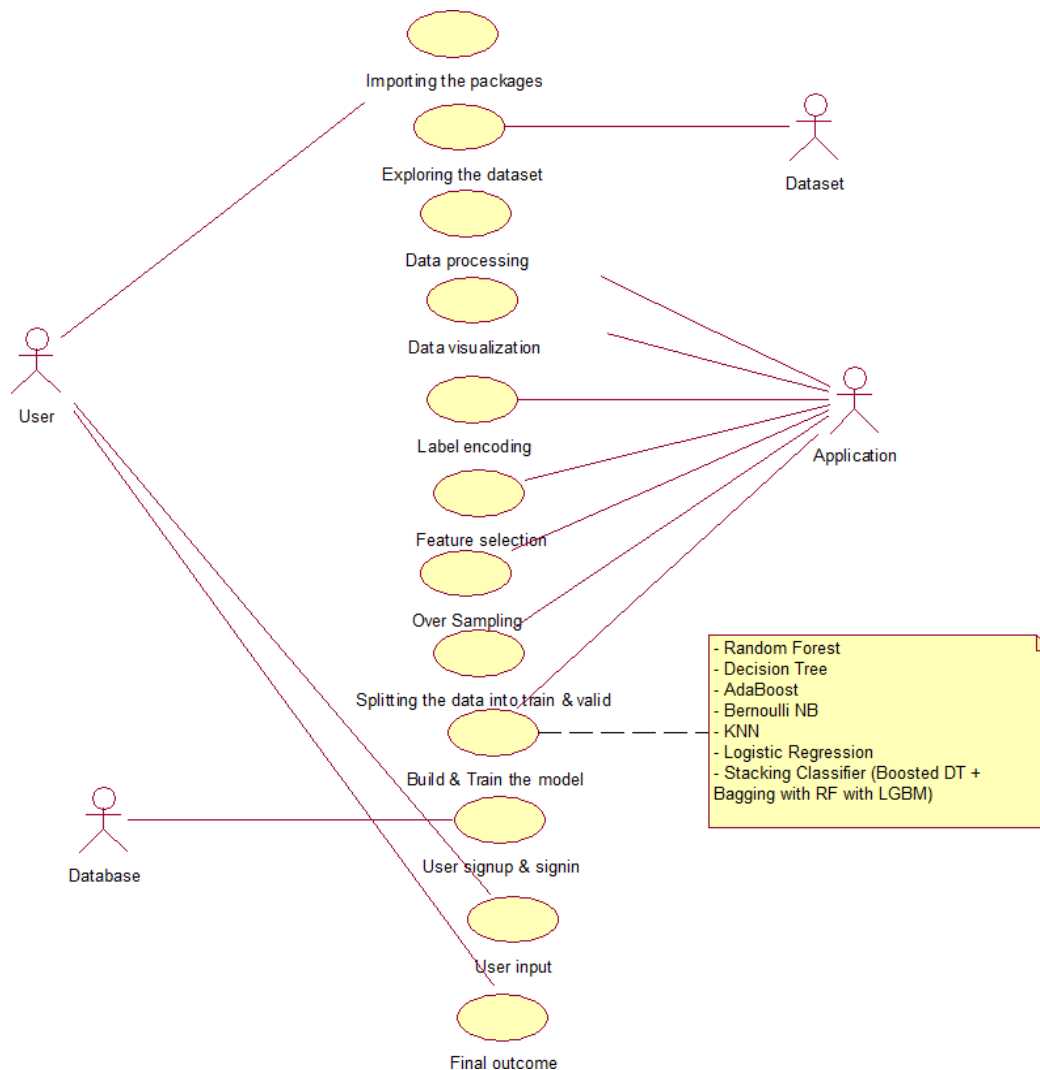


Fig.5.2.1 USE CASE DIAGRAM

Class Diagram

The class diagram describes the structure of the automated NID system by representing classes such as User, Model, and Dataset. Each class has attributes and operations while aggregation relationships demonstrate how these classes interact, aiding in system design.

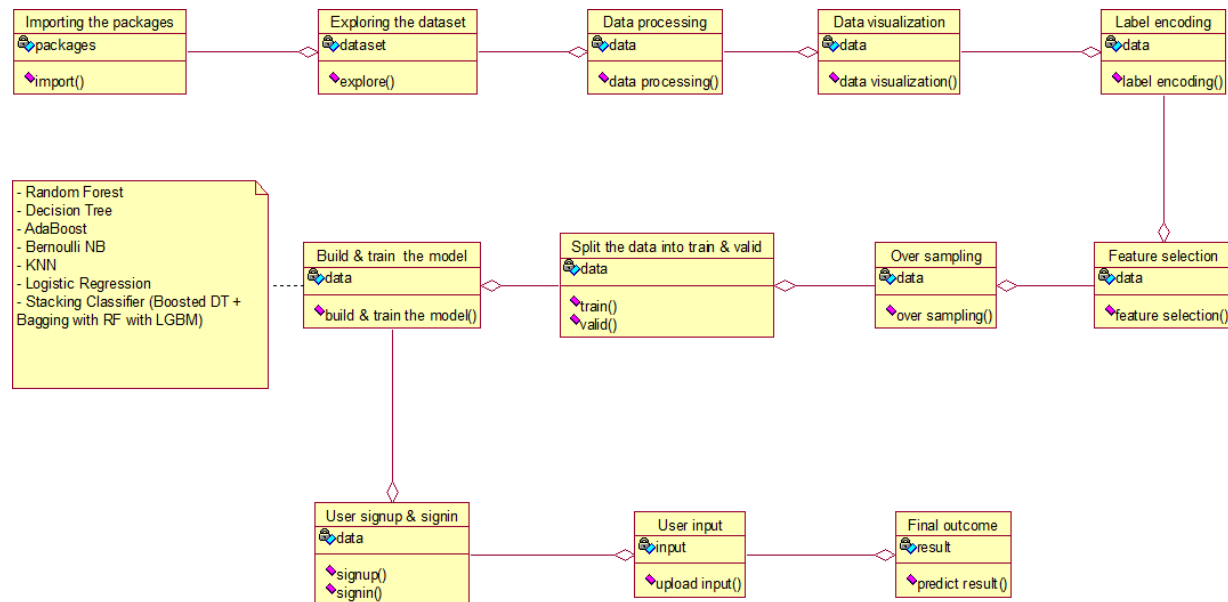


Fig.5.2.2 CLASS DIAGRAM

Activity Diagram

An activity diagram illustrates the workflow of processes in the NID system, detailing actions like data visualization and oversampling. Decisions guide control flow between activities, enabling a clear understanding of process sequences and user interactions throughout the automated intrusion detection framework.

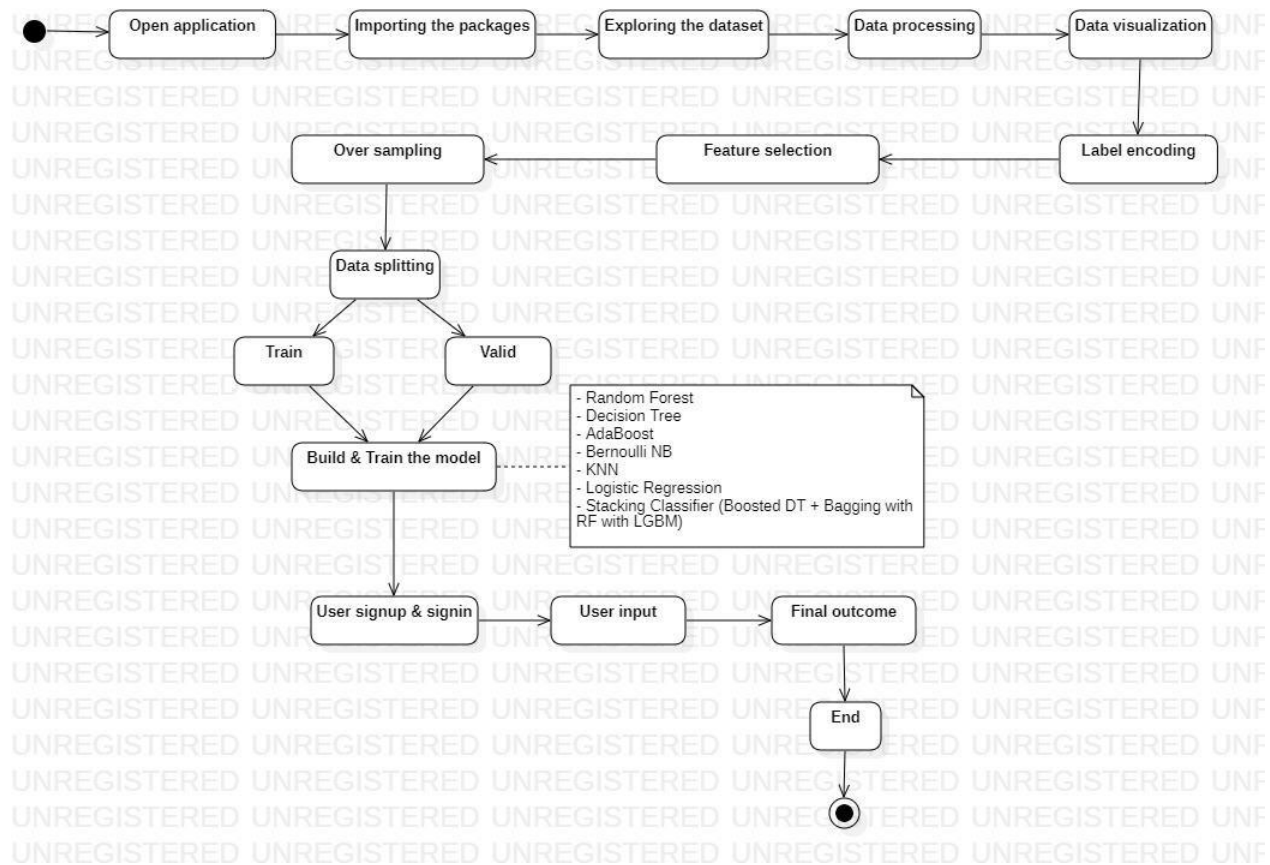


Fig.5.2.3 ACTIVITY DIAGRAM

Sequence Diagram

The sequence diagram captures the interactions between objects over time, showcasing messages exchanged during processes like user input and model training. By detailing object lifetimes and the sequence of object messages, it clarifies the flow of operations within the automated NID system.

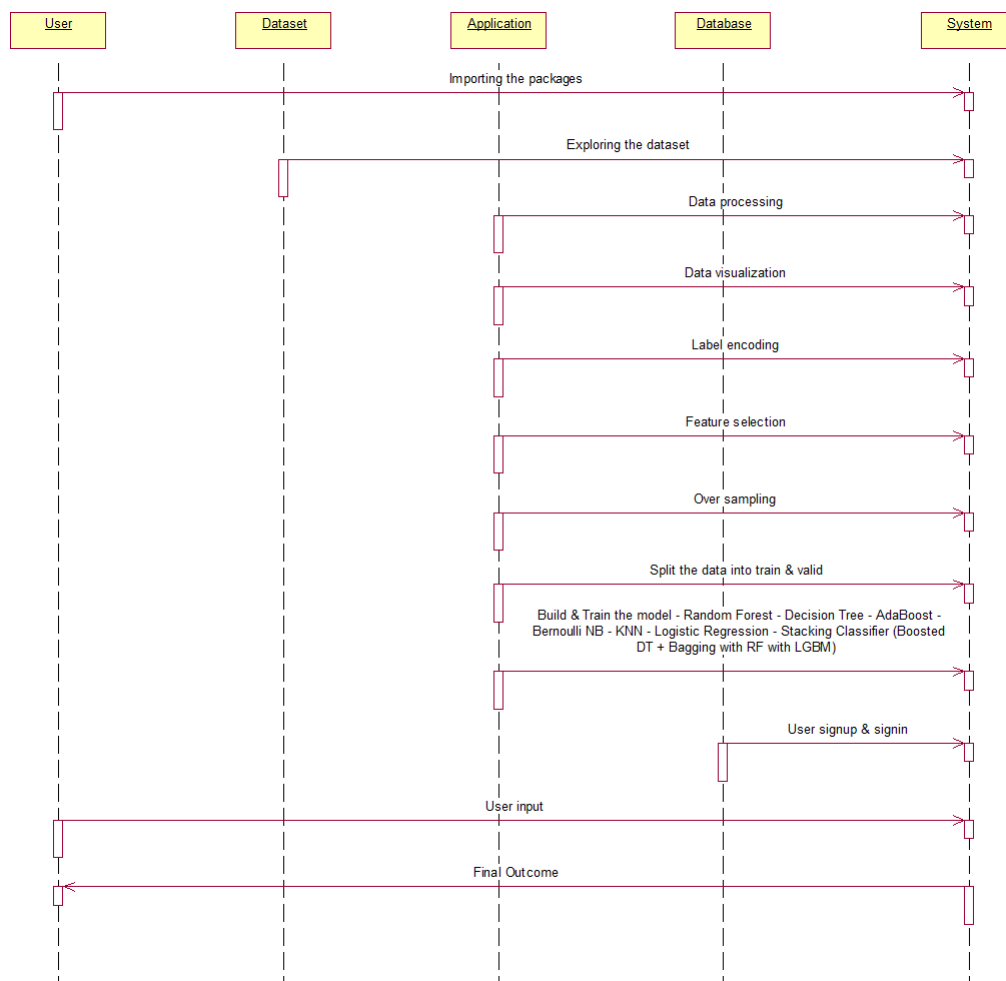


Fig.5.2.4 SEQUENCE DIAGRAM

Collaboration Diagram

A collaboration diagram outlines how objects interact to achieve specific tasks, emphasizing the links between objects such as User, Model, and Dataset. Link messages indicate the sequence of interactions, providing insights into the system's collaborative behavior during processes like data processing and final outcome generation.

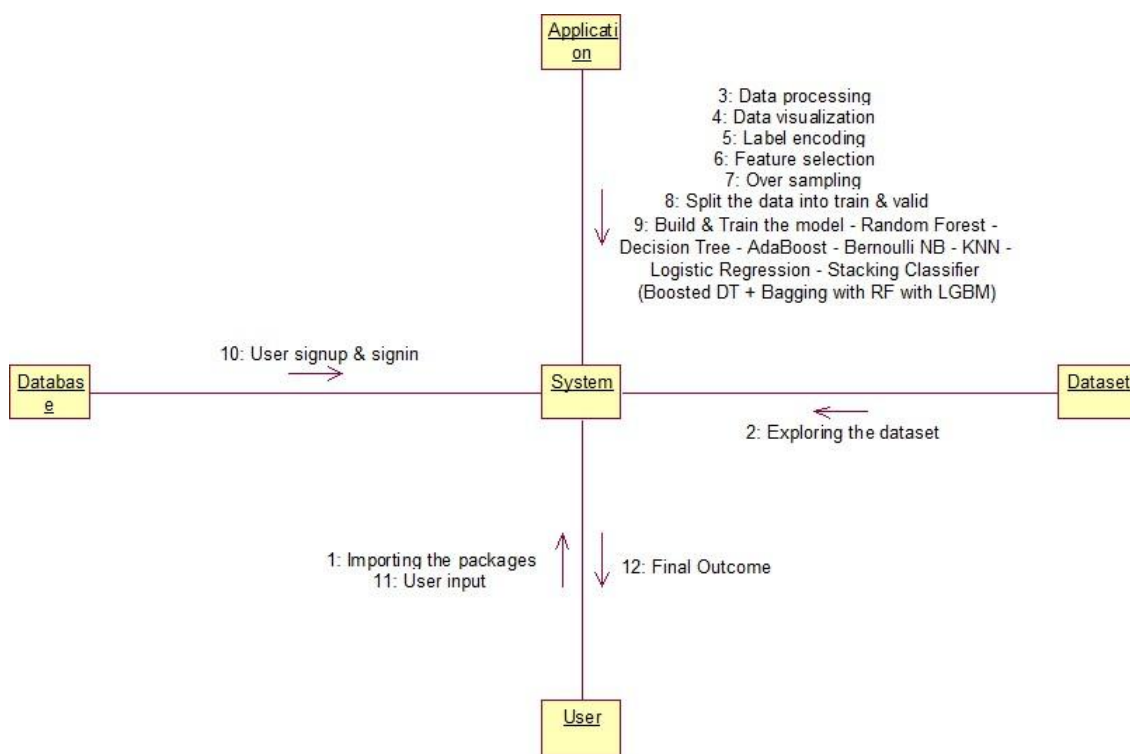


Fig.5.2.5 COLLABORATION DIAGRAM

Component Diagram

The component diagram depicts the system's high-level structure by representing components like the User Interface, NID Engine, and Database. Packages group related components

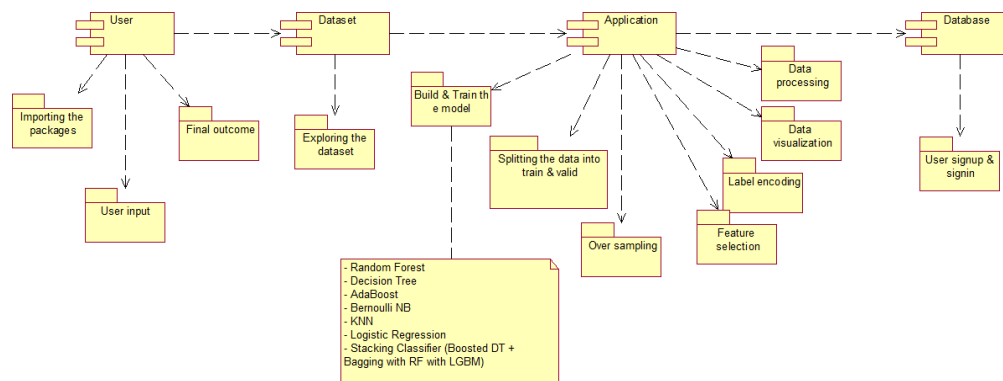


Fig.5.2.6 COMPONENT DIAGRAM

Deployment Diagram

The deployment diagram shows the physical deployment of the NID system across nodes, such as servers and user devices. Connectors illustrate communication paths between nodes

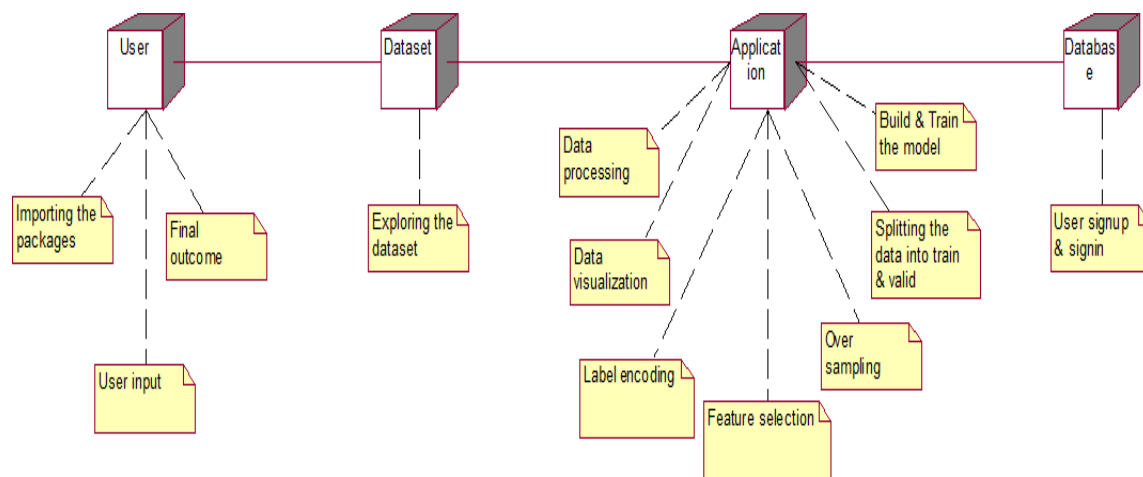


Fig.5.2.7 DEPLOYMENT DIAGRAM

CHAPTER 6:IMPLEMENTATION

MODULES:

- **Data loading:** using this module we are going to import the dataset.
- **Data Processing:** Data processing involves removing duplicate data, applying drop cleaning techniques, and normalizing values to ensure uniformity across the dataset. This step is crucial for enhancing data quality and preparing it for effective analysis and modeling in the NID system.
- **Data Visualization:** Data visualization employs various techniques to graphically represent the processed data, enabling a clearer understanding of patterns, trends, and relationships. Through visual tools like charts and graphs, stakeholders can gain insights into intrusion detection metrics and system performance.
- **Label Encoding:** Label encoding transforms string categorical variables into integer representations, facilitating the model's understanding of input data. By converting these values, the system can process categorical information effectively, improving the overall performance and accuracy of the intrusion detection algorithms.
- **Feature Selection:** Feature selection identifies and retains the most relevant features for the intrusion detection model, employing techniques such as Filter Method, Wrapper Method, and an Embedded approach combining Lasso with Random Forest and Recursive Feature Elimination (RFE) for optimal feature representation.
- **Oversampling:** Oversampling involves using the Synthetic Minority Over-sampling Technique (SOMTE) to balance the dataset by increasing the representation of minority classes. This process enhances the model's ability to detect rare intrusions, improving its performance and reducing bias during training.
- **Splitting data into train & valid:** using this module data will be divided into train & valid

- **Model generation:** Model building - Random Forest - Decision Tree - AdaBoost - Bernoulli NB - KNN - Logistic Regression - Stacking Classifier (Boosted DT + Bagging with RF with LGBM). Performance evaluation metrics for each algorithm is calculated.
- **User signup & login:** Using this module will get registration and login
- **User input:** Using this module will give input for prediction
- **Prediction:** final predicted displayed

Extension:

We extended the original research by integrating an ensemble method that combines predictions from multiple models, including a Stacking Classifier, to enhance data analysis accuracy. This approach leverages the strengths of various feature selection methods and machine learning models, leading to a more robust prediction system. Additionally, we will develop a front end using the Flask framework to facilitate user testing and implement user authentication for enhanced security.

Advantages:

1. Combining multiple model predictions through stacking improves overall accuracy, providing a more comprehensive approach to data analysis and attack detection.
2. Utilizing diverse feature selection methods allows for better model training, helping to identify the most relevant features impacting predictions.
3. The Stacking Classifier enhances the ensemble model's performance by learning from the outputs of base models to improve final predictions.
4. A Flask-based front end offers an intuitive interface for users, enabling seamless interaction with the system while maintaining secure user access.

Algorithms:

Random Forest: is a machine learning method that creates many decision trees and combines their results to improve accuracy and reduce errors. In a Network Intrusion Detection (NID) system

Decision Tree: is a simple model that splits data based on specific conditions, forming a tree-like structure to make decisions. In NID systems, it clearly classifies network traffic and helps quickly detect threats or unusual activity in IoT networks.

AdaBoost: or Adaptive Boosting, improves performance by combining several weak models into one strong model. In intrusion detection, it focuses on correcting errors made in earlier steps, which helps it catch more hidden or subtle attacks.

Bernoulli Naive Bayes: is a fast and simple model that works well with data that has binary (yes/no) features. It checks if certain features are present or not to decide if network traffic is normal or malicious, making it suitable for real-time detection.

K-Nearest Neighbors (KNN): works by comparing new network data to past data and looking at the closest matches. It helps find unusual patterns or anomalies, which could be signs of attacks. It's easy to use and adapts well to changing data.

Logistic Regression: is used to predict whether something will happen or not (like an intrusion). It gives a probability score for each event, helping security teams decide which alerts need urgent attention.

Stacking Classifier: is a powerful method that combines different models like Random Forest, AdaBoost, and LightGBM. It uses the strengths of each model to improve detection accuracy and make the intrusion detection system more reliable and effective.

6.2 SAMPLE CODE:

```
import numpy as np
import pandas as pd
import warnings
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow.keras import regularizers
import xgboost as xgb
from sklearn.decomposition import PCA
from sklearn import tree
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import RobustScaler
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn import metrics
pd.set_option('display.max_columns',None)
warnings.filterwarnings('ignore')
%matplotlib inline
# Read Train and Test dataset
data_train = pd.read_csv("../input/nslkdd/KDDTrain+.txt")
# Check data
data_train.head()
```

```
columns =
(['duration','protocol_type','service','flag','src_bytes','dst_bytes','land','wrong_fragment','urgent','h
ot'

,'num_failed_logins','logged_in','num_compromised','root_shell','su_attempted','num_root','num_
file_creations'

,'num_shells','num_access_files','num_outbound_cmds','is_host_login','is_guest_login','count','sr
v_count','error_rate'

,'srv_error_rate','error_rate','srv_error_rate','same_srv_rate','diff_srv_rate','srv_diff_host_rate','
dst_host_count','dst_host_srv_count'

,'dst_host_same_srv_rate','dst_host_diff_srv_rate','dst_host_same_src_port_rate','dst_host_srv_di
ff_host_rate','dst_host_error_rate'

,'dst_host_srv_error_rate','dst_host_error_rate','dst_host_srv_error_rate','outcome','level'])

# Assign name for columns

data_train.columns = columns

data_train.head()

data_train.info()

data_train.describe().style.background_gradient(cmap='Blues').set_properties(**{'font-
family':'Segoe UI'})

data_train.loc[data_train['outcome'] == "normal", "outcome"] = 'normal'

data_train.loc[data_train['outcome'] != 'normal', "outcome"] = 'attack'

def pie_plot(df, cols_list, rows, cols):

    fig, axes = plt.subplots(rows, cols)

    for ax, col in zip(axes.ravel(), cols_list):

        df[col].value_counts().plot(ax=ax, kind='pie', figsize=(15, 15), fontsize=10,
autopct='%1.0f%%')

        ax.set_title(str(col), fontsize = 12)

    plt.show()

pie_plot(data_train, ['protocol_type', 'outcome'], 1, 2)

def Scaling(df_num, cols):

    std_scaler = RobustScaler()

    std_scaler_temp = std_scaler.fit_transform(df_num)
```

```
std_df = pd.DataFrame(std_scaler_temp, columns=cols)
return std_df

cat_cols = ['is_host_login', 'protocol_type', 'service', 'flag', 'land', 'logged_in', 'is_guest_login', 'level',
'outcome']

def preprocess(dataframe):
    df_num = dataframe.drop(cat_cols, axis=1)
    num_cols = df_num.columns
    scaled_df = Scaling(df_num, num_cols)

    dataframe.drop(labels=num_cols, axis="columns", inplace=True)
    dataframe[num_cols] = scaled_df[num_cols]

    dataframe.loc[dataframe['outcome'] == "normal", "outcome"] = 0
    dataframe.loc[dataframe['outcome'] != 0, "outcome"] = 1

    dataframe = pd.get_dummies(dataframe, columns = ['protocol_type', 'service', 'flag'])
    return dataframe

scaled_train = preprocess(data_train)
x = scaled_train.drop(['outcome', 'level'], axis = 1).values
y = scaled_train['outcome'].values
y_reg = scaled_train['level'].values

pca = PCA(n_components=20)
pca = pca.fit(x)
x_reduced = pca.transform(x)

print("Number of original features is {} and of reduced features is {}".format(x.shape[1],
x_reduced.shape[1]))

y = y.astype('int')
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

x_train_reduced, x_test_reduced, y_train_reduced, y_test_reduced = train_test_split(x_reduced, y,
test_size=0.2, random_state=42)

x_train_reg, x_test_reg, y_train_reg, y_test_reg = train_test_split(x, y_reg, test_size=0.2,
random_state=42)

kernal_evals = dict()

def evaluate_classification(model, name, X_train, X_test, y_train, y_test):

    train_accuracy = metrics.accuracy_score(y_train, model.predict(X_train))

    test_accuracy = metrics.accuracy_score(y_test, model.predict(X_test))


    train_precision = metrics.precision_score(y_train, model.predict(X_train))

    test_precision = metrics.precision_score(y_test, model.predict(X_test))


    train_recall = metrics.recall_score(y_train, model.predict(X_train))

    test_recall = metrics.recall_score(y_test, model.predict(X_test))


    kernal_evals[str(name)] = [train_accuracy, test_accuracy, train_precision, test_precision,
train_recall, test_recall]

    print("Training Accuracy " + str(name) + " {} Test Accuracy ".format(train_accuracy*100) +
str(name) + " {}".format(test_accuracy*100))

    print("Training Precesion " + str(name) + " {} Test Precesion ".format(train_precision*100) +
str(name) + " {}".format(test_precision*100))

    print("Training Recall " + str(name) + " {} Test Recall ".format(train_recall*100) + str(name)
+ " {}".format(test_recall*100))


    actual = y_test

    predicted = model.predict(X_test)

    confusion_matrix = metrics.confusion_matrix(actual, predicted)

    cm_display = metrics.ConfusionMatrixDisplay(confusion_matrix = confusion_matrix,
display_labels = ['normal', 'attack'])
```

```
fig, ax = plt.subplots(figsize=(10,10))
ax.grid(False)
cm_display.plot(ax=ax)
lr = LogisticRegression().fit(x_train, y_train)
evaluate_classification(lr, "Logistic Regression", x_train, x_test, y_train, y_test)
knn = KNeighborsClassifier(n_neighbors=20).fit(x_train, y_train)
evaluate_classification(knn, "KNeighborsClassifier", x_train, x_test, y_train, y_test)
gnb = GaussianNB().fit(x_train, y_train)
evaluate_classification(gnb, "GaussianNB", x_train, x_test, y_train, y_test)
lin_svc = svm.LinearSVC().fit(x_train, y_train)
evaluate_classification(lin_svc, "Linear SVC(LBasedImpl)", x_train, x_test, y_train, y_test)
dt = DecisionTreeClassifier(max_depth=3).fit(x_train, y_train)
tdt = DecisionTreeClassifier().fit(x_train, y_train)
evaluate_classification(tdt, "DecisionTreeClassifier", x_train, x_test, y_train, y_test)
def f_importances(coef, names, top=-1):
    imp = coef
    imp, names = zip(*sorted(list(zip(imp, names))))

    # Show all features
    if top == -1:
        top = len(names)

    plt.figure(figsize=(10,10))
    plt.barh(range(top), imp[::-1][0:top], align='center')
    plt.yticks(range(top), names[::-1][0:top])
    plt.title('feature importances for Decision Tree')
    plt.show()
```

```
features_names = data_train.drop(['outcome', 'level'], axis = 1)
f_importances(abs(tdt.feature_importances_), features_names, top=18)
fig = plt.figure(figsize=(15,12))
tree.plot_tree(dt, filled=True)
rf = RandomForestClassifier().fit(x_train, y_train)
evaluate_classification(rf, "RandomForestClassifier", x_train, x_test, y_train, y_test)
f_importances(abs(rf.feature_importances_), features_names, top=18)
xg_r = xgb.XGBRegressor(objective='reg:linear', n_estimators = 20).fit(x_train_reg, y_train_reg)
name = "XGBOOST"
train_error = metrics.mean_squared_error(y_train_reg, xg_r.predict(x_train_reg), squared=False)
test_error = metrics.mean_squared_error(y_test_reg, xg_r.predict(x_test_reg), squared=False)
print("Training Error " + str(name) + " {} Test error ".format(train_error) + str(name) + " {}".format(test_error))
y_pred = xg_r.predict(x_test_reg)
df = pd.DataFrame({"Y_test": y_test_reg, "Y_pred": y_pred})
plt.figure(figsize=(16,8))
plt.plot(df[:80])
plt.legend(['Actual', 'Predicted'])
rrf = RandomForestClassifier().fit(x_train_reduced, y_train_reduced)
evaluate_classification(rrf, "PCA RandomForest", x_train_reduced, x_test_reduced, y_train_reduced, y_test_reduced)
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=64, activation='relu', input_shape=(x_train.shape[1:]),
        kernel_regularizer=regularizers.L1L2(l1=1e-5, l2=1e-4),
        bias_regularizer=regularizers.L2(1e-4),
        activity_regularizer=regularizers.L2(1e-5)),
    tf.keras.layers.Dropout(0.4),
    tf.keras.layers.Dense(units=128, activation='relu',
        kernel_regularizer=regularizers.L1L2(l1=1e-5, l2=1e-4),
```

```
        bias_regularizer=regularizers.L2(1e-4),
        activity_regularizer=regularizers.L2(1e-5)),
tf.keras.layers.Dropout(0.4),
tf.keras.layers.Dense(units=512, activation='relu',
        kernel_regularizer=regularizers.L1L2(l1=1e-5, l2=1e-4),
        bias_regularizer=regularizers.L2(1e-4),
        activity_regularizer=regularizers.L2(1e-5)),
tf.keras.layers.Dropout(0.4),
tf.keras.layers.Dense(units=128, activation='relu',
        kernel_regularizer=regularizers.L1L2(l1=1e-5, l2=1e-4),
        bias_regularizer=regularizers.L2(1e-4),
        activity_regularizer=regularizers.L2(1e-5)),
tf.keras.layers.Dropout(0.4),
tf.keras.layers.Dense(units=1, activation='sigmoid'),
])
model.compile(optimizer='adam', loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),
metrics=['accuracy'])
model.summary()
from keras.utils.vis_utils import plot_model
plot_model(model, to_file='model_plot.png', show_shapes=True, show_layer_names=True)
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=10, verbose=1)
plt.plot(history.history['loss'], label='loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.xlabel('Epoch')
plt.ylabel('SCCE Loss')
plt.legend()
plt.grid(True)
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label='val_accuracy')
```

```
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
keys = [key for key in kernal_evals.keys()]
values = [value for value in kernal_evals.values()]
fig, ax = plt.subplots(figsize=(20, 6))
ax.bar(np.arange(len(keys)) - 0.2, [value[0] for value in values], color='darkred', width=0.25,
align='center')
ax.bar(np.arange(len(keys)) + 0.2, [value[1] for value in values], color='y', width=0.25,
align='center')
ax.legend(["Training Accuracy", "Test Accuracy"])
ax.set_xticklabels(keys)
ax.set_xticks(np.arange(len(keys)))
plt.ylabel("Accuracy")
plt.show()
keys = [key for key in kernal_evals.keys()]
values = [value for value in kernal_evals.values()]
fig, ax = plt.subplots(figsize=(20, 6))
ax.bar(np.arange(len(keys)) - 0.2, [value[2] for value in values], color='g', width=0.25,
align='center')
ax.bar(np.arange(len(keys)) + 0.2, [value[3] for value in values], color='b', width=0.25,
align='center')
ax.legend(["Training Precesion", "Test Presision"])
ax.set_xticklabels(keys)
ax.set_xticks(np.arange(len(keys)))
plt.ylabel("Precesion")
plt.show()
keys = [key for key in kernal_evals.keys()]
values = [value for value in kernal_evals.values()]
```

```
fig, ax = plt.subplots(figsize=(20, 6))

ax.bar(np.arange(len(keys)) - 0.2, [value[2] for value in values], color='g', width=0.25,
align='center')

ax.bar(np.arange(len(keys)) + 0.2, [value[3] for value in values], color='b', width=0.25,
align='center')

ax.legend(["Training Recall", "Test Recall"])

ax.set_xticklabels(keys)

ax.set_xticks(np.arange(len(keys)))

plt.ylabel("Recall")

plt.show()
```

CHAPTER 7: SOFTWARE ENVIRONMENT

Cybersecurity:

Cyber security means protecting a company, its people, and its data from online threats. As cyberattacks become more frequent and advanced, and company networks become bigger and more complex, different security tools are needed to reduce the risk of attacks.

The Different Types of Cybersecurity

1. **NetworkSecurity**

Keeps the company's network safe. It controls who can access data and blocks bad files or hackers from entering the system.

2. **CloudSecurity**

Protects data and apps stored on the internet (cloud). It stops hackers from stealing or damaging cloud information.

3. **EndpointSecurity**

Secures devices like laptops and desktops. This is important when people work from different places.

4. **MobileSecurity**

Protects phones and tablets from harmful apps or messages. Only safe devices are allowed to use company data.

5. **IoTSecurity**

Keeps smart devices (like cameras and sensors) safe. It finds weak devices and blocks hackers from using them.

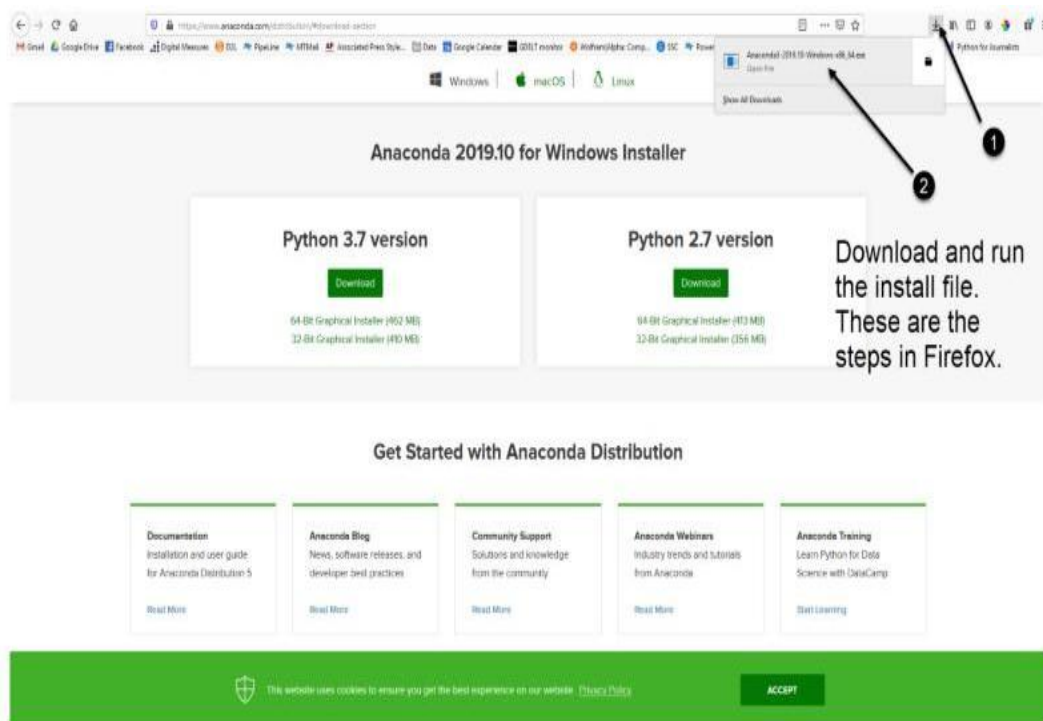
6. **ZeroTrust**

This means “don't trust anything automatically.” It checks every person and device before giving access, even if they are inside the company.



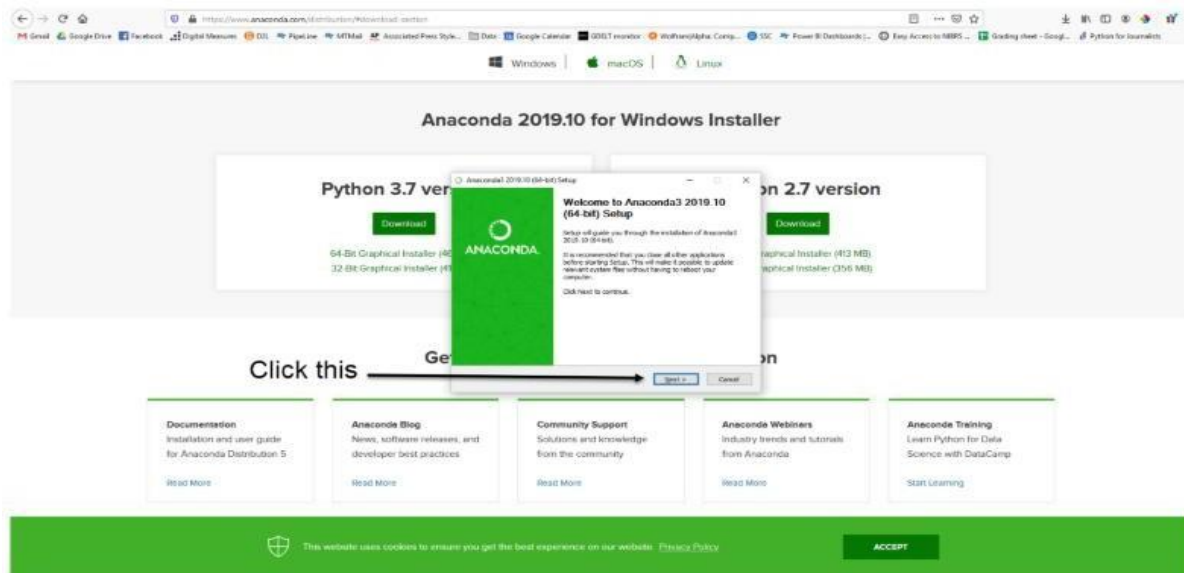
Python Anaconda Installation

1. Go to this link and download Anaconda for Windows, Mac, or Linux: – [Download anaconda](#)

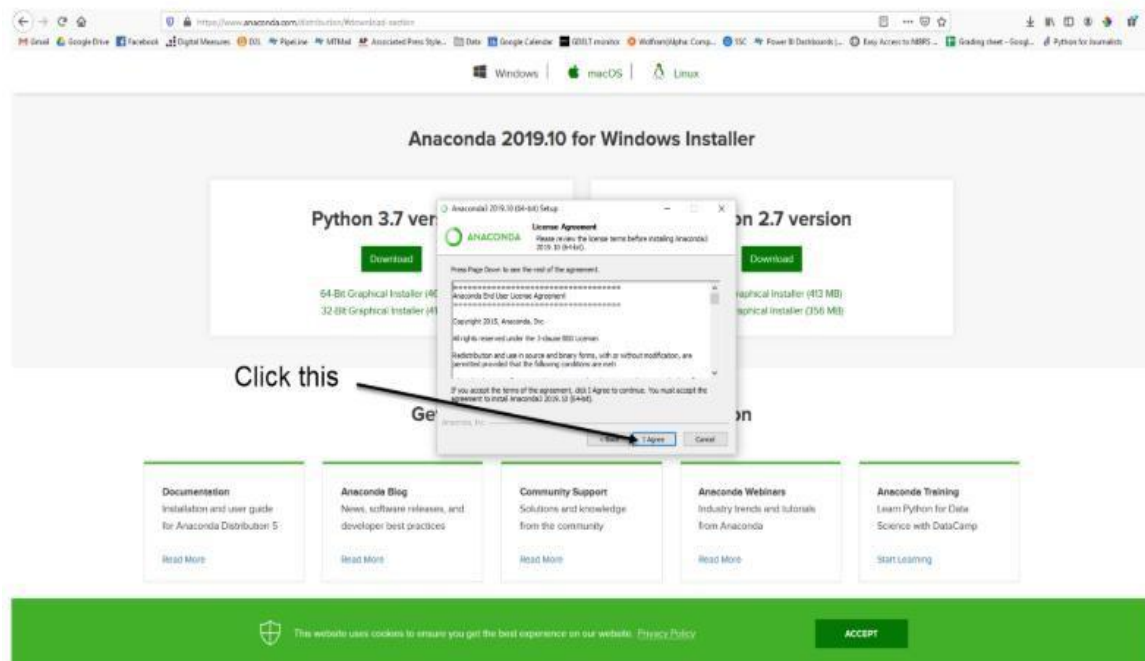


You can download the installer for Python 3.7 or for Python 2.7 (at the time of writing). And you can download it for a 32-bit or 64-bit machine.

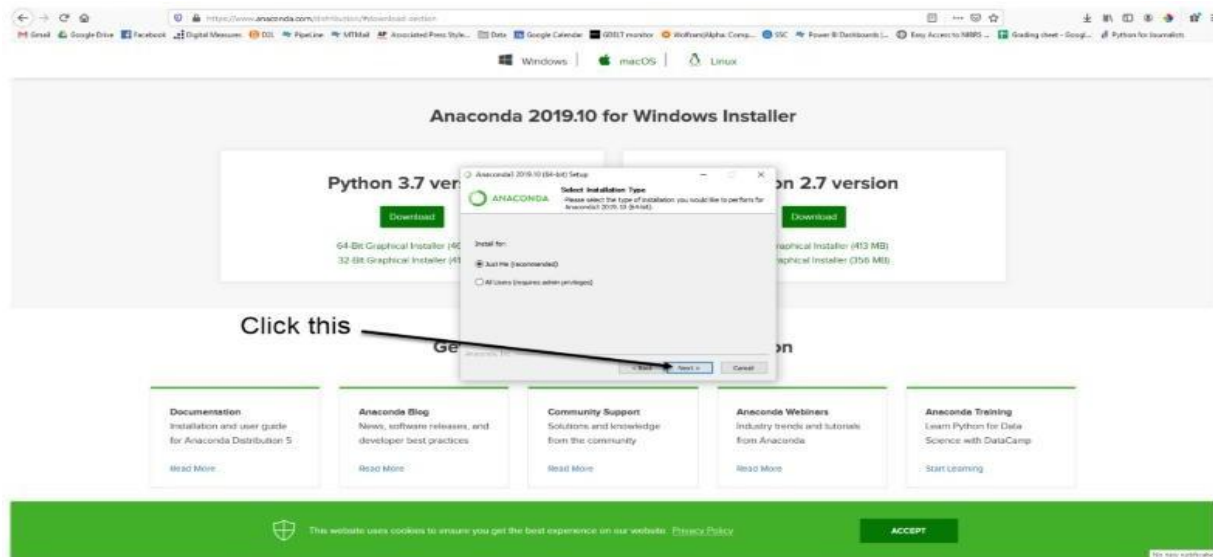
2. Click on the downloaded .exe to open it. This is the Anaconda setup. Click next.



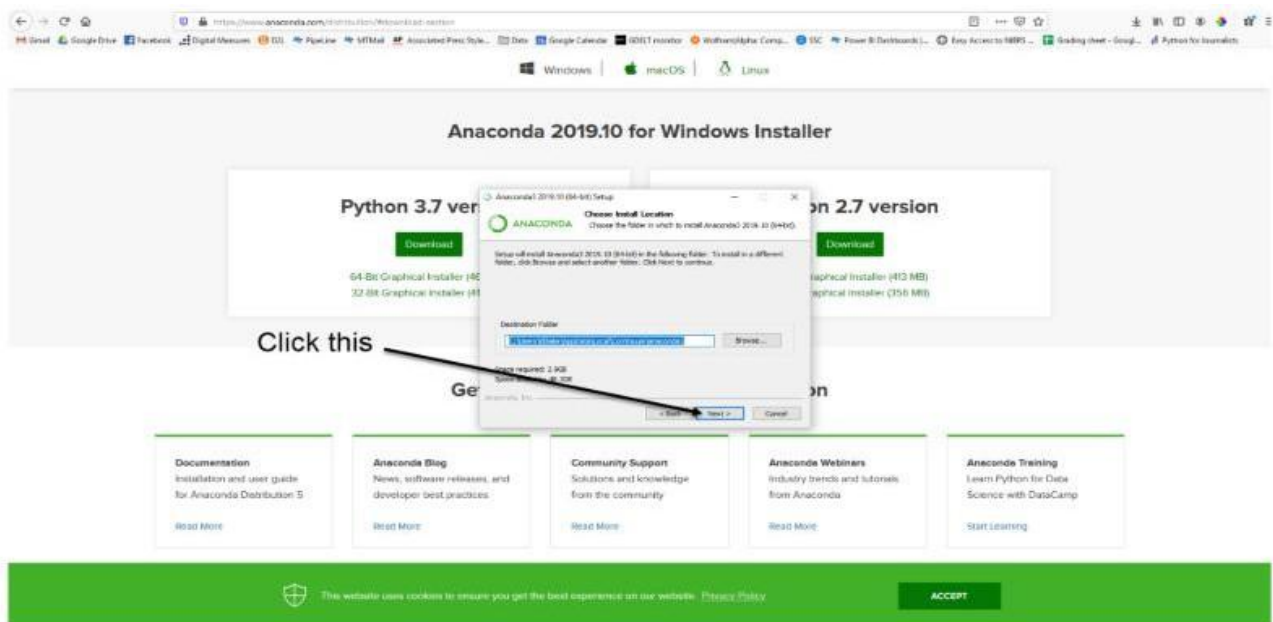
3. Now, you'll see the license agreement. Click on 'I Agree'.



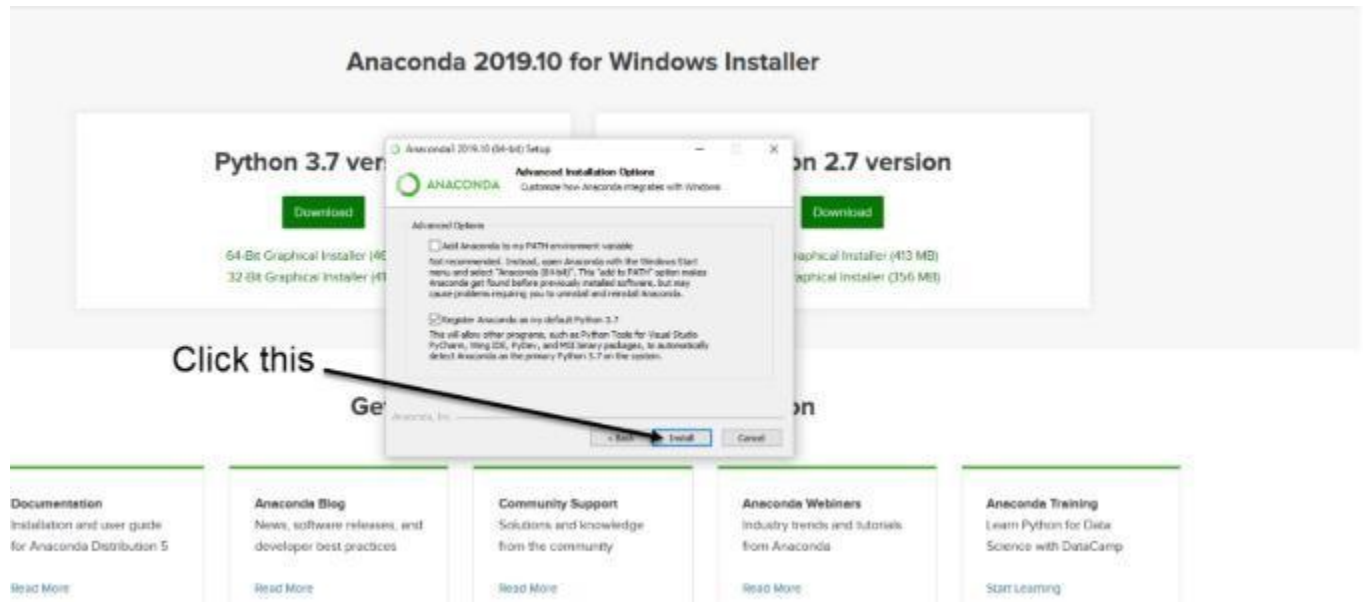
4. You can install it for all users or just for yourself. If you want to install it for all users, you need administrator privileges.



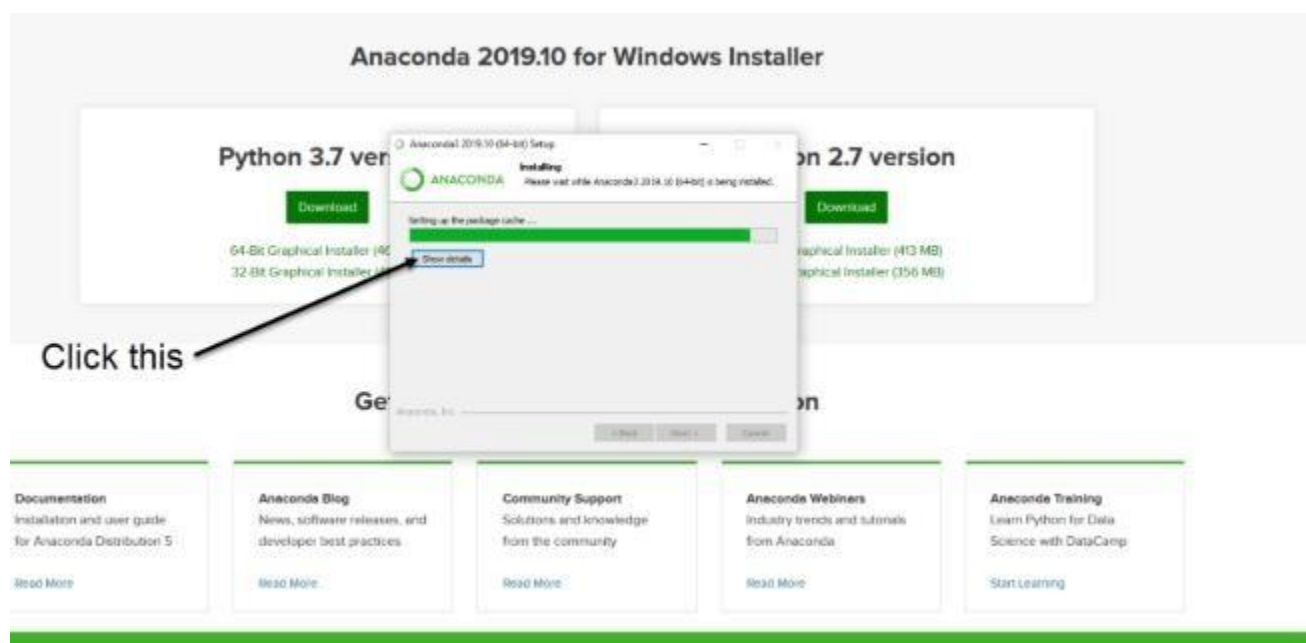
5. Choose where you want to install it. Here, you can see the available space and how much you need.



6. Now, you'll get some advanced options. You can add Anaconda to your system's PATH environment variable, and register it as the primary system Python 3.7. If you add it to PATH, it will be found before any other installation. Click on 'Install'.



7. It will unpack some packages and extract some files on your machine. This will take a few minutes.



The installation will take a while. When it finishes, click this.

Anaconda 2019.10 for Windows Installer

Python 3.7 version

Download

64-Bit Graphical Installer (412 MB)
32-Bit Graphical Installer (156 MB)

Python 2.7 version

Download

64-Bit Graphical Installer (412 MB)
32-Bit Graphical Installer (156 MB)

Anaconda 2019.10 (64-bit) Setup

ANACONDA

Anaconda and Jupyter are working together to bring you Anaconda-powered environments tightly integrated in the Python IDE.

Python for Anaconda is available at:
<https://www.anaconda.com/python>

Click this

Next >

Documentation
Installation and user guide for Anaconda Distribution 5
[Read More](#)

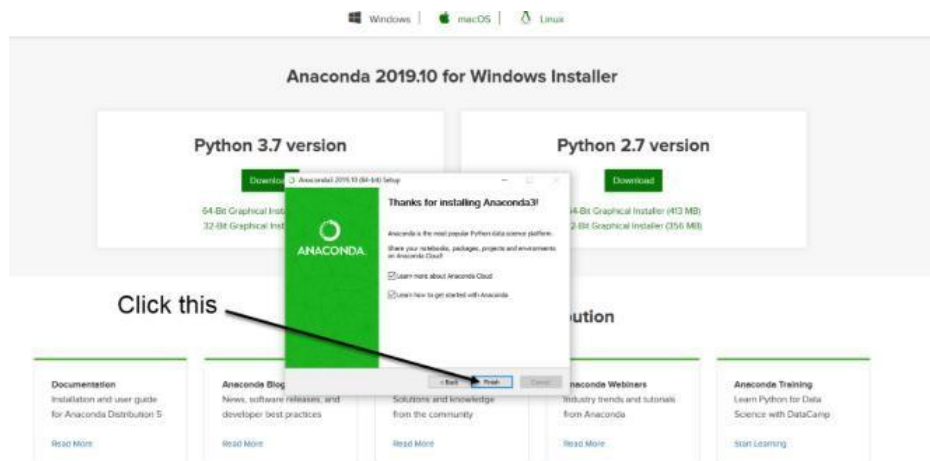
Anaconda Blog
News, software releases, and developer best practices
[Read More](#)

Solutions and knowledge from the community
[Read More](#)

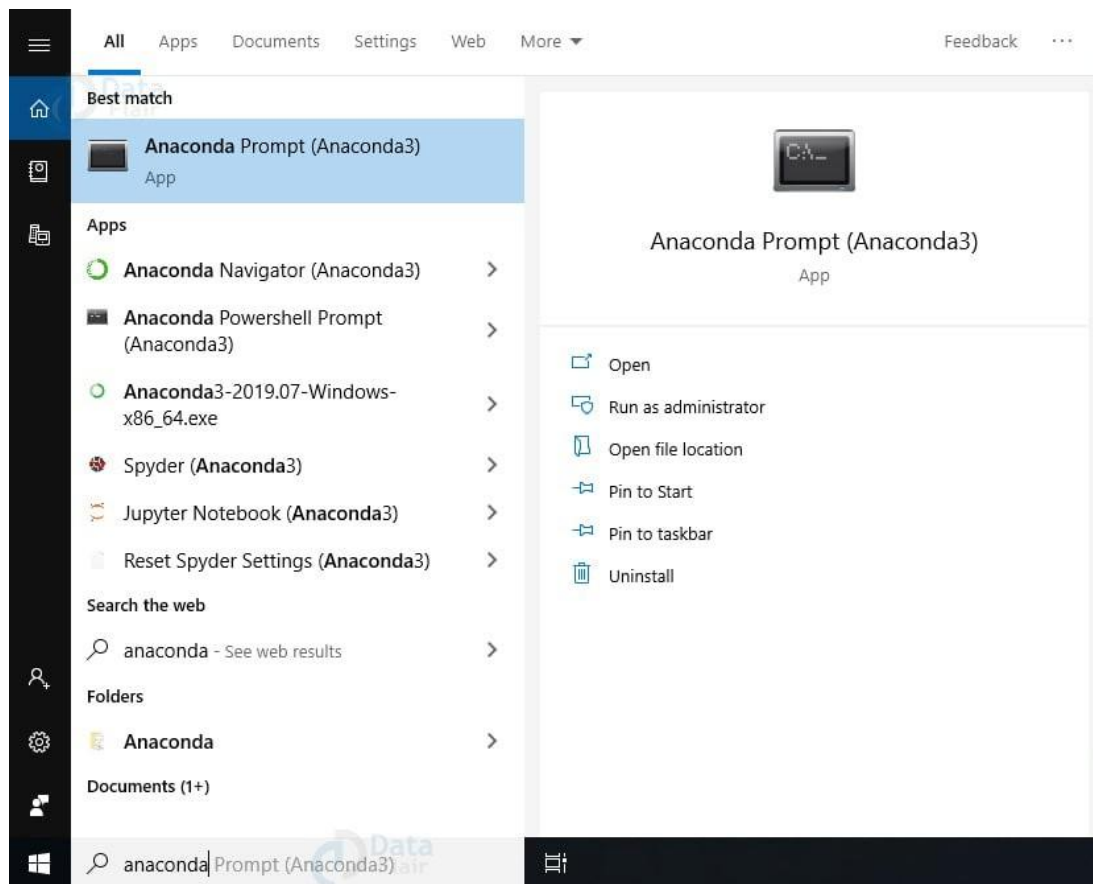
Anaconda Webinars
Industry trends and tutorials from Anaconda
[Read More](#)

Anaconda Training
Learn Python for Data Science with DataCamp
[Start Learning](#)

10. The installation is complete. You can choose to get more information about Anaconda cloud and how to get started with Anaconda. Click Finish.



11. If you search for Anaconda now, you will see the following



LIBRARIES/PACKGES :-

Scikit-learn

A free machine learning library in Python used for tasks like classification, regression, clustering, model selection, and data preprocessing.

NumPy

A basic library for scientific computing in Python. It supports large arrays, matrices, and fast math functions. It helps perform complex math easily.

TensorFlow

An open-source library created by Google for building and training deep learning and machine learning models. It is powerful and used by researchers and developers.

Keras

A simple, user-friendly Python library for deep learning. It runs on top of TensorFlow and makes building neural networks easy and fast.

Pandas

A popular library for handling and analyzing data in Python. It helps work with tables and large datasets quickly and easily.

Matplotlib

A Python library used to create charts and graphs like line plots, bar charts, and pie charts with just a few lines of code.

SciPy

An open-source library for scientific and technical computing. It is used for tasks like math, statistics, optimization, and data processing.

CHAPTER 8. SYSTEM TESTING

System testing, also referred to as system-level testing or system integration testing, is the process in which a quality assurance (QA) team evaluates how the various components of an application interact together in the full, integrated system or application.

8.1 Types of system testing

System testing helps the QA team check if the application meets all its technical, business, and functional needs. To do this, they use different testing methods to make sure everything is covered and to find major issues that could affect how the app works before it is released...

The following are the common types of system testing techniques:

Performance Testing checks how fast, stable, and reliable a system is under different conditions. It looks at things like load time and how well the system handles pressure.

Usability Testing checks if the system is easy and comfortable for users to use. It measures how many mistakes users make, how quickly they complete tasks, and how satisfied they are.

Load Testing checks how the system works when many users or heavy tasks are happening at the same time. It measures speed, response time, and user capacity.

Regression Testing makes sure that after making changes or updates, the old features still work properly and no new bugs have appeared.

Migration Testing ensures data and functions move smoothly from an old system to a new one without any loss or issues.

Scalability Testing checks if the system can handle more (or fewer) users or tasks when needed, as user demand changes.

Functionality Testing confirms that the system works as expected and meets all the business and technical requirements.

Recovery Testing checks if the system can recover properly after crashes, failures, or errors.



Phases of system testing

System testing checks if the whole application works properly as one unit. It is done after testing each part separately and together. If everything works well, the app gets one last check (acceptance testing) before it is given to users. Any problems found are noted and fixed.

8.2 TEST CASES:

S.NO	INPUT	If available	If not available
1	User signup	User get registered into the application	There is no process
2	User signin	User get login into the application	There is no process
3	Enter input for prediction	Prediction result displayed	There is no process

CHAPTER 9:SCREENSHOTS

Tables:

Performance Evaluation Table Filter Method FS

ML Model	Accuracy	Precision	Recall	F1_score
Random Forest	0.752	0.758	0.752	0.751
Decision Tree	0.601	0.715	0.601	0.619
AdaBoost	0.529	0.656	0.529	0.567
BernoulliNB	0.170	0.994	0.170	0.283
KNN	0.652	0.754	0.652	0.660
Logistic Regression	0.594	0.637	0.594	0.598
Extension Voting Classifier	0.797	0.804	0.797	0.797

Performance Evaluation Table Wrapper Based FS

ML Model	Accuracy	Precision	Recall	F1_score
Random Forest	0.871	0.871	0.871	0.869
Decision Tree	0.604	0.773	0.604	0.648
AdaBoost	0.668	0.699	0.668	0.666
BernoulliNB	0.296	0.862	0.296	0.436
KNN	0.799	0.800	0.799	0.797
Logistic Regression	0.174	0.702	0.174	0.269
Extension Voting Classifier	0.954	0.955	0.954	0.953

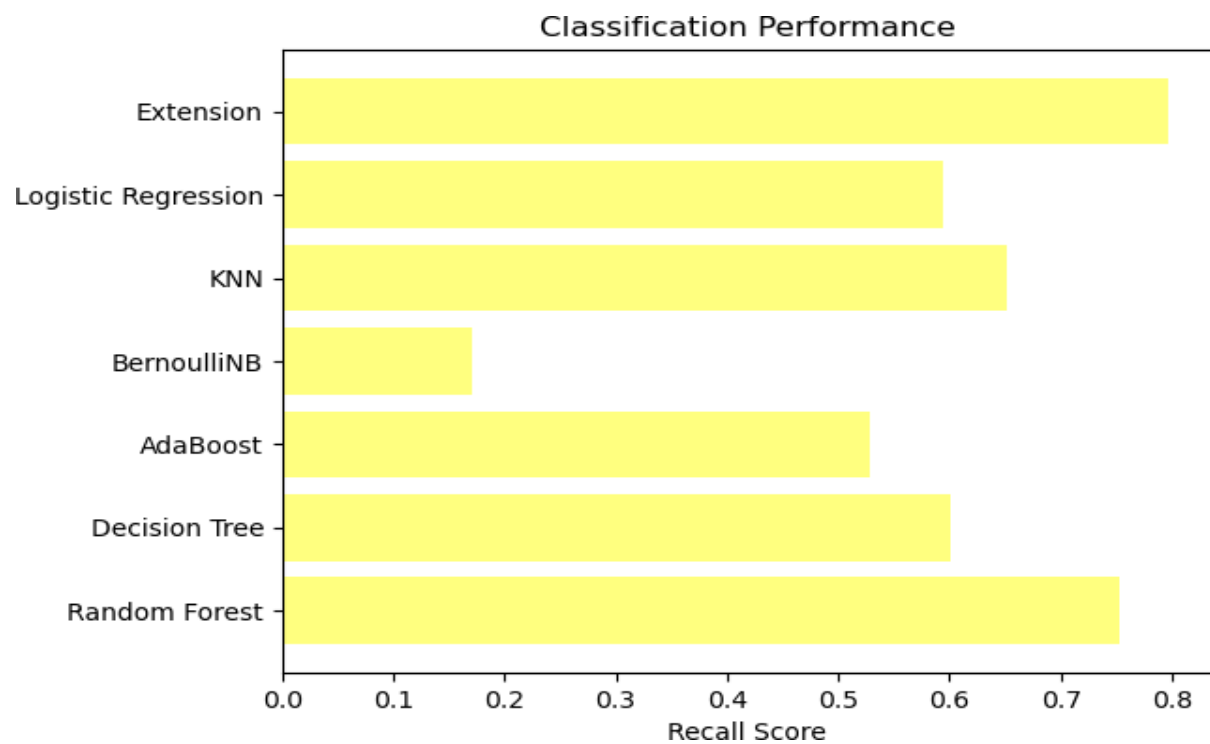
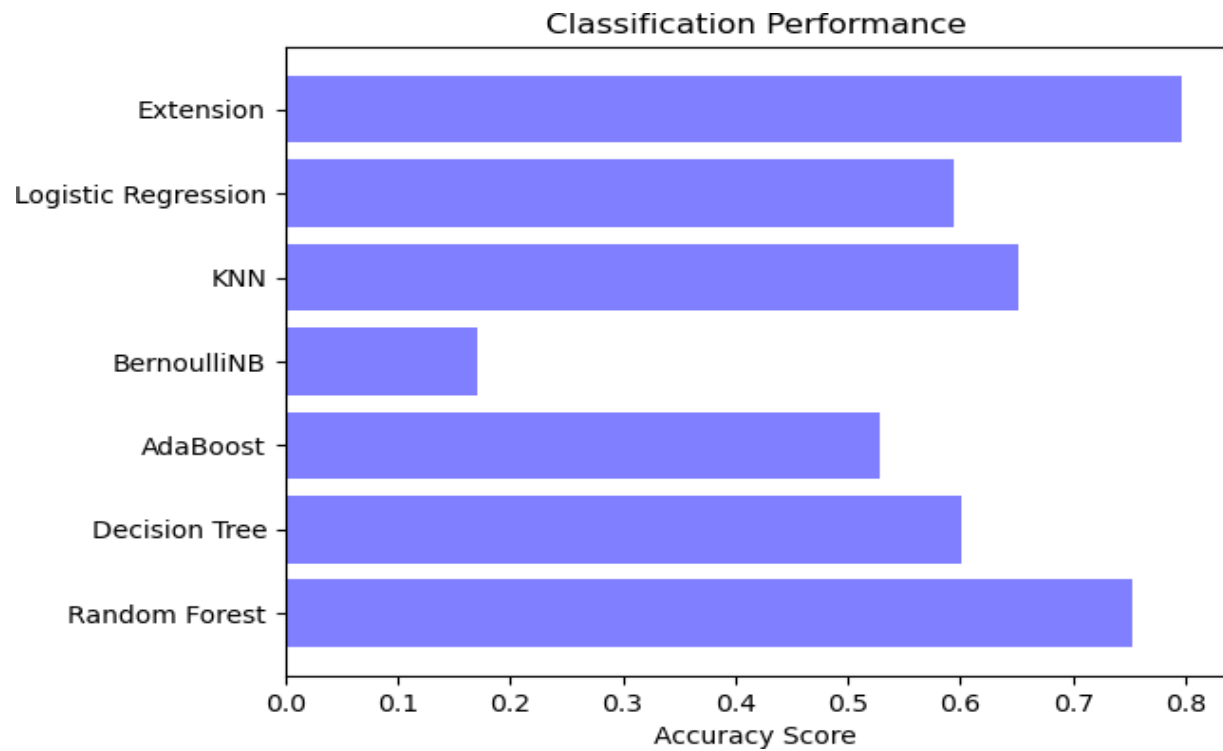
Performance Evaluation Table Embedded Method FS

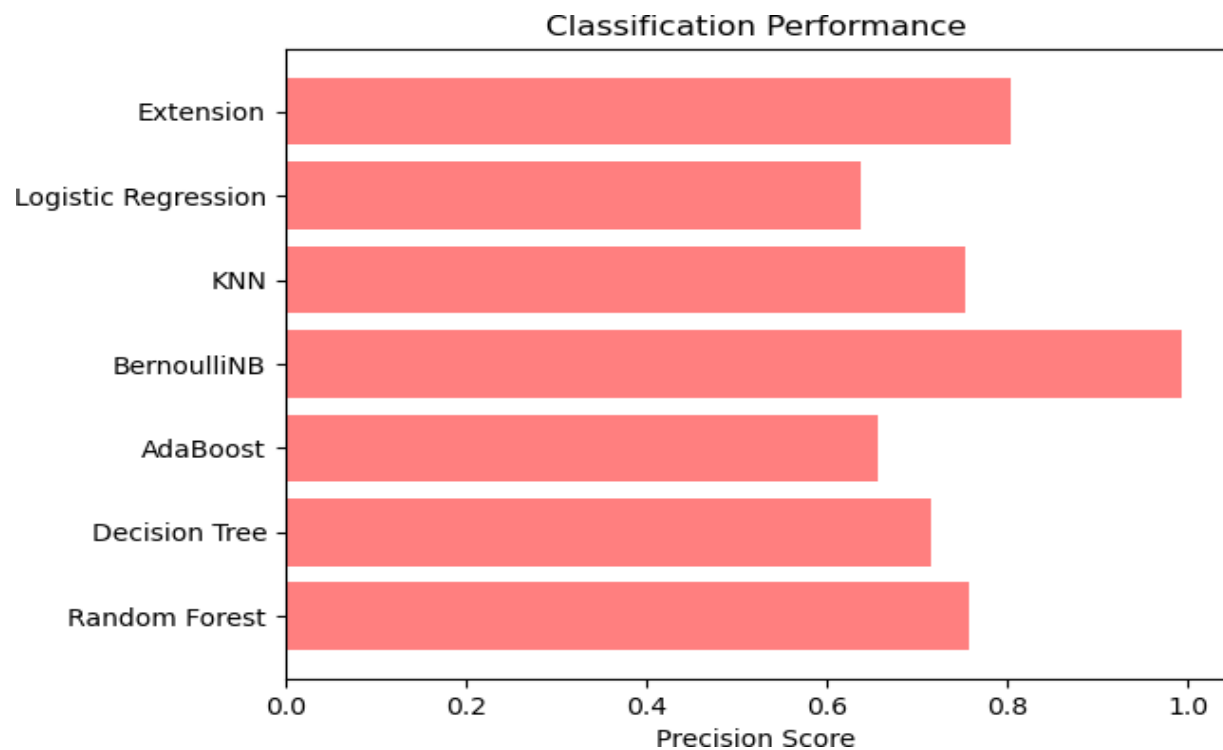
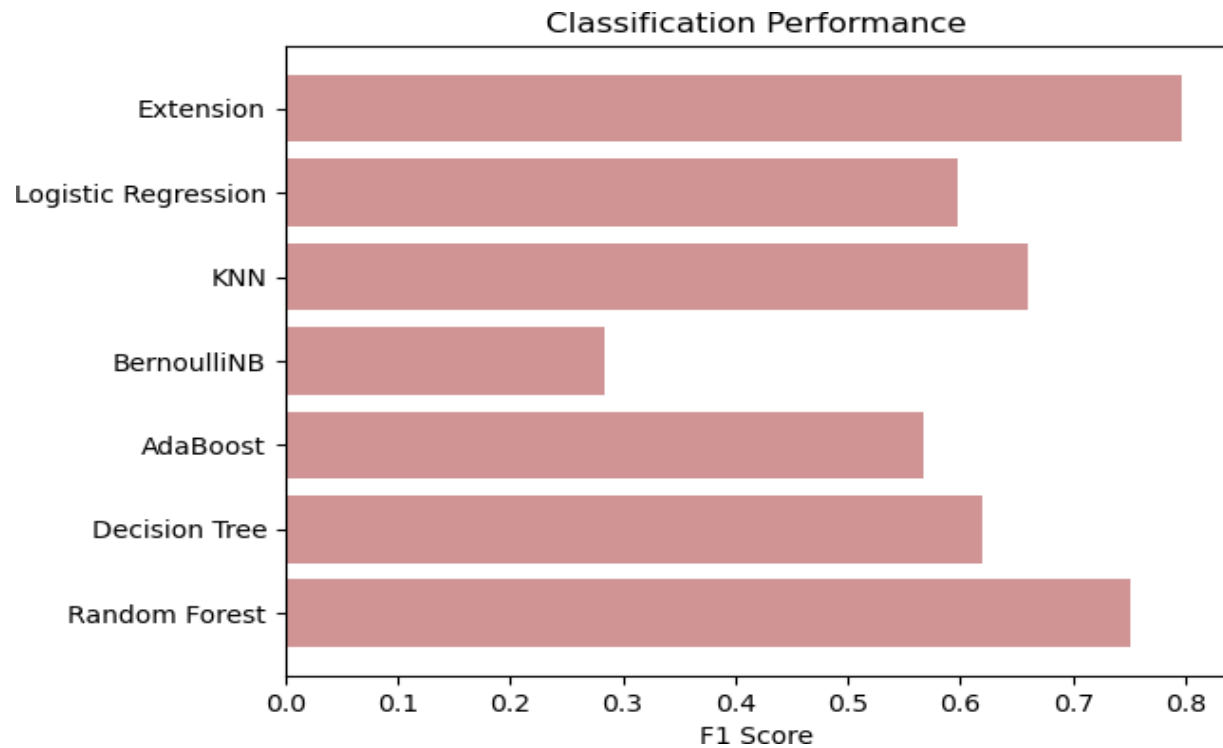
ML Model	Accuracy	Precision	Recall	F1_score
Random Forest	0.836	0.835	0.836	0.835
Decision Tree	0.578	0.703	0.578	0.603
AdaBoost	0.647	0.665	0.647	0.652
BernoulliNB	0.298	0.888	0.298	0.444
KNN	0.724	0.725	0.724	0.721
Logistic Regression	0.500	0.592	0.500	0.516
Extension Voting Classifier	0.957	0.957	0.957	0.956

Performance Evaluation Table PCC FS

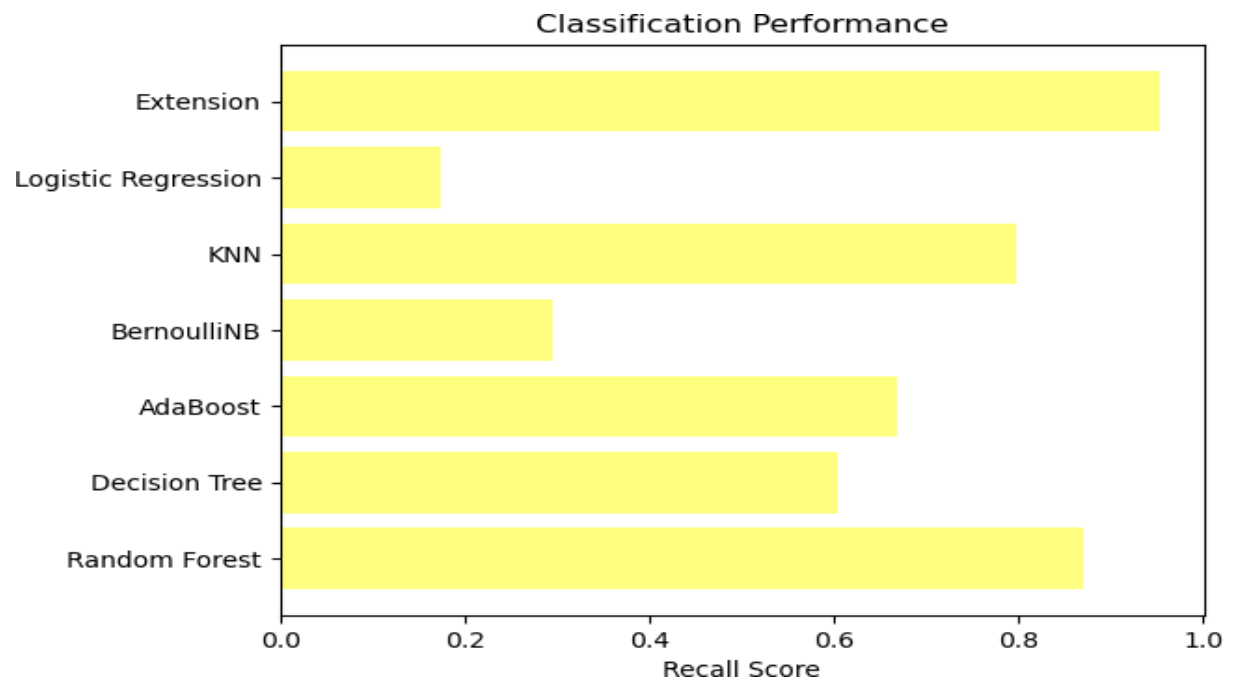
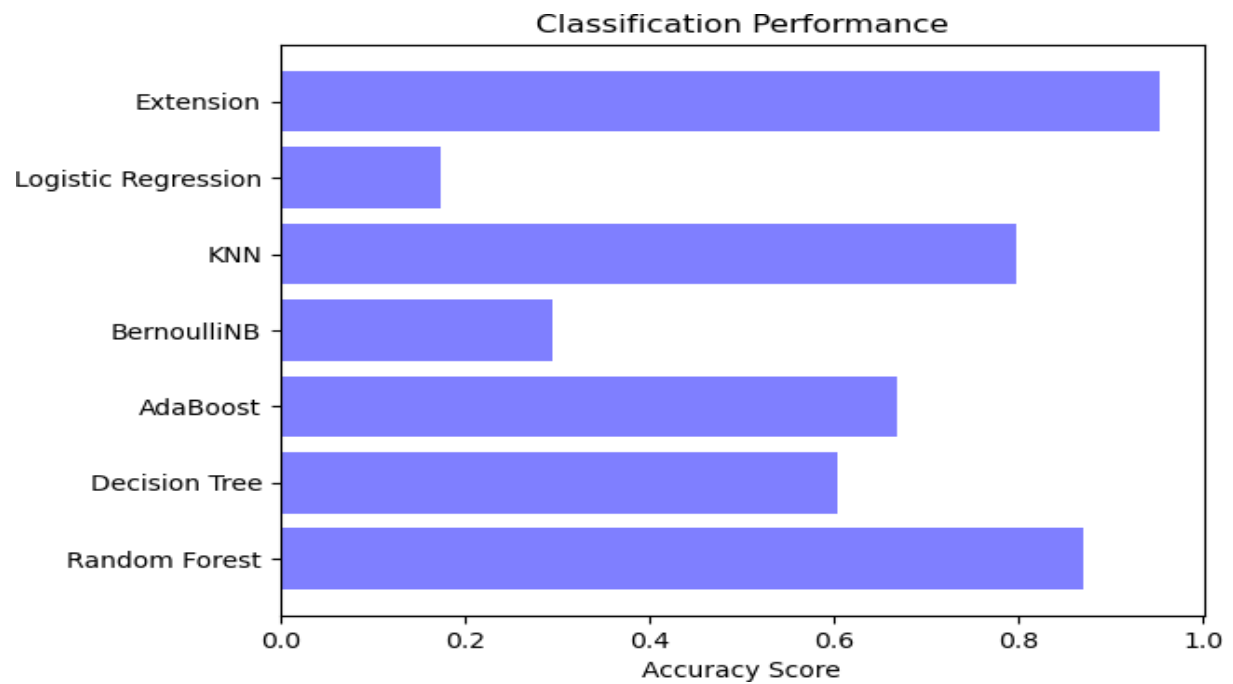
ML Model	Accuracy	Precision	Recall	F1_score
Random Forest	0.871	0.874	0.871	0.870
Decision Tree	0.601	0.740	0.601	0.621
AdaBoost	0.680	0.705	0.680	0.686
BernoulliNB	0.282	0.818	0.282	0.417
KNN	0.843	0.844	0.843	0.841
Logistic Regression	0.633	0.697	0.633	0.653
Extension Voting Classifier	0.939	0.941	0.939	0.939

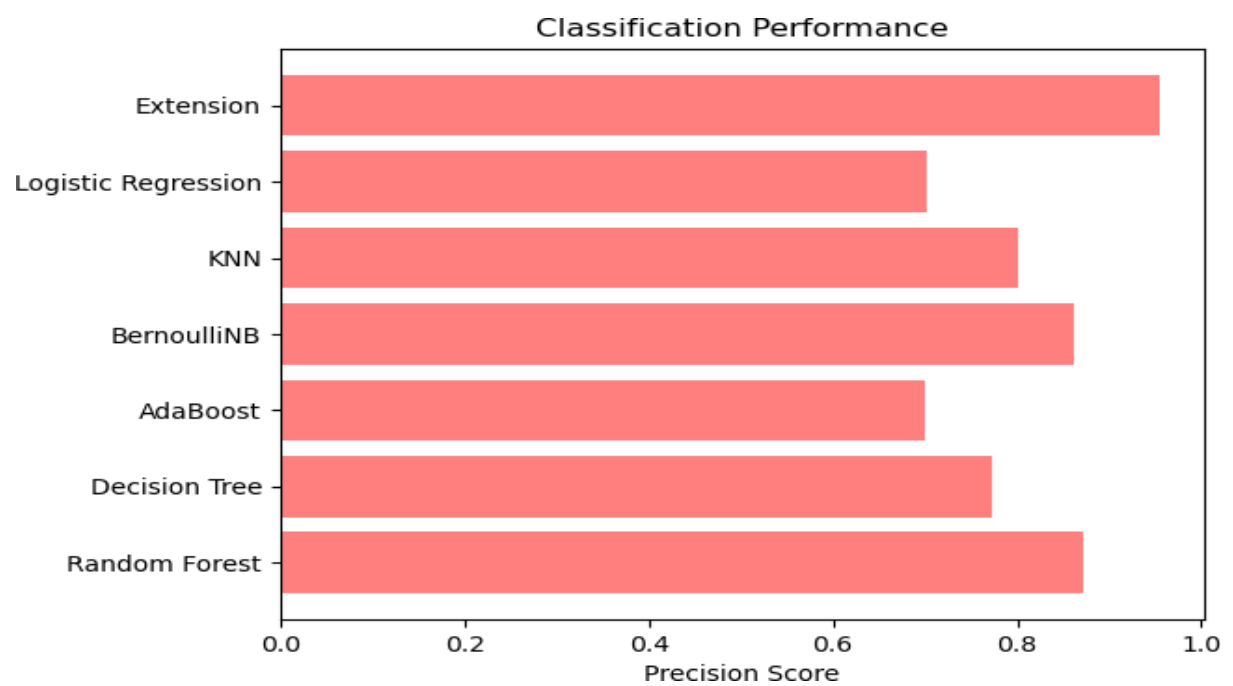
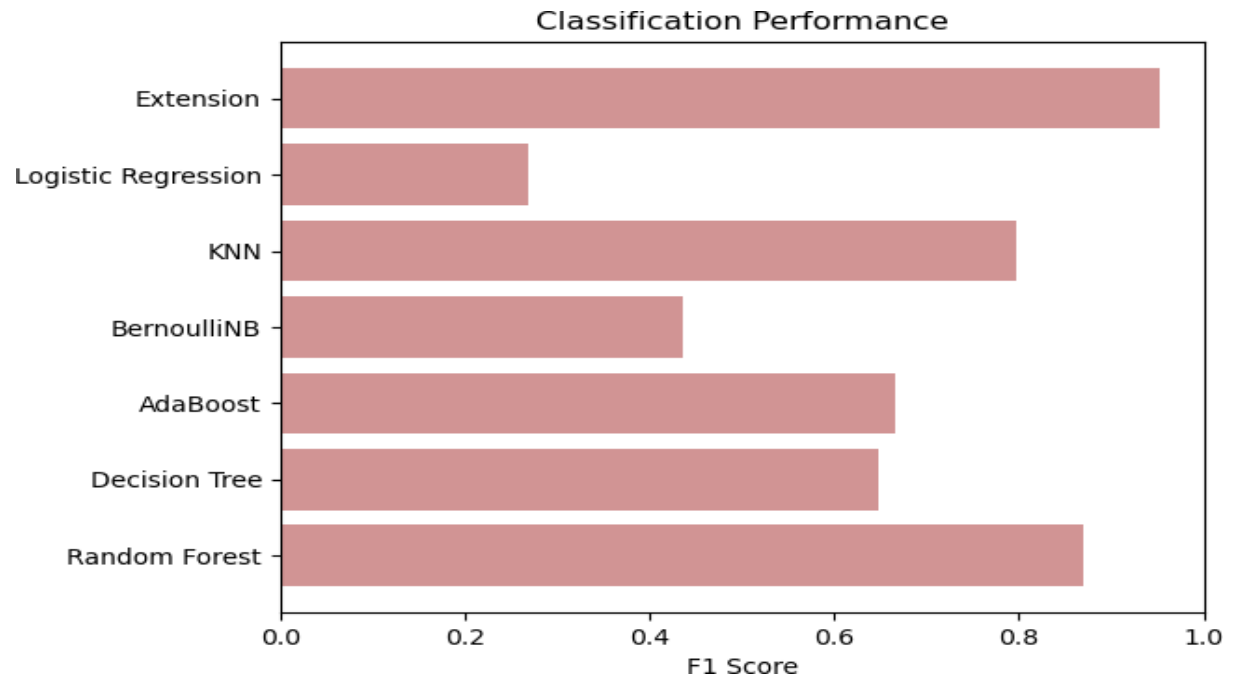
Graphs:



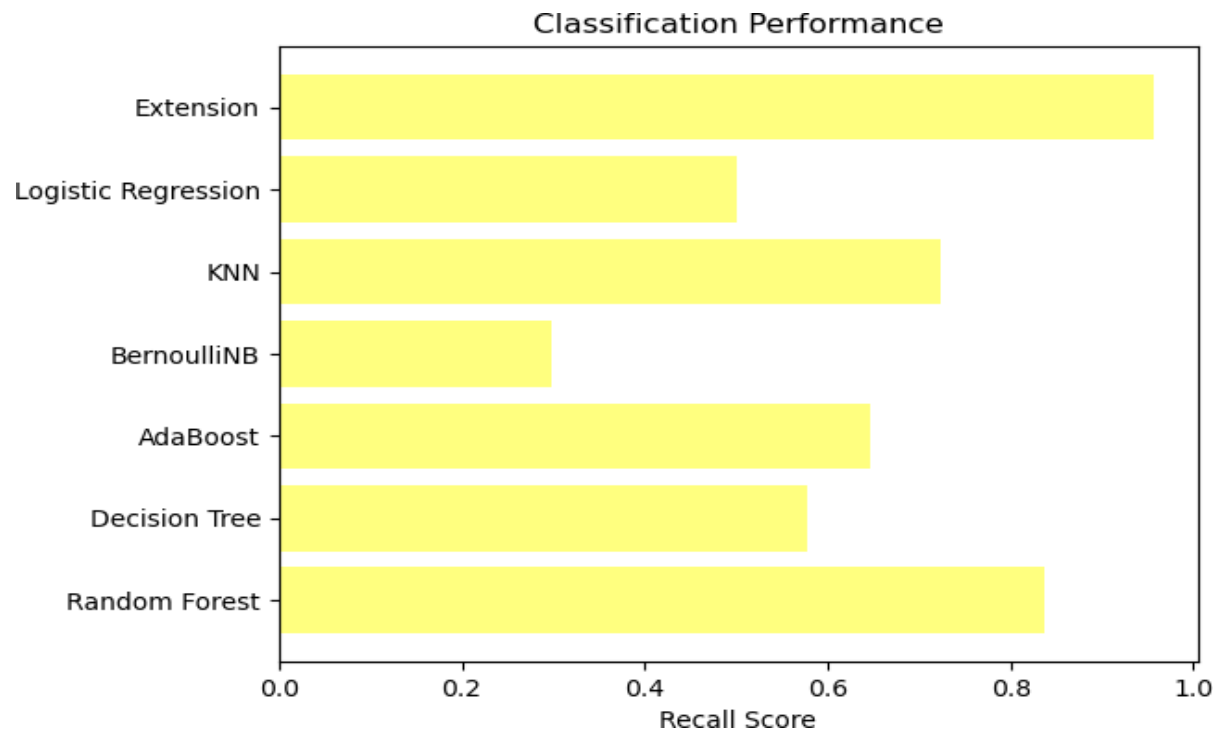
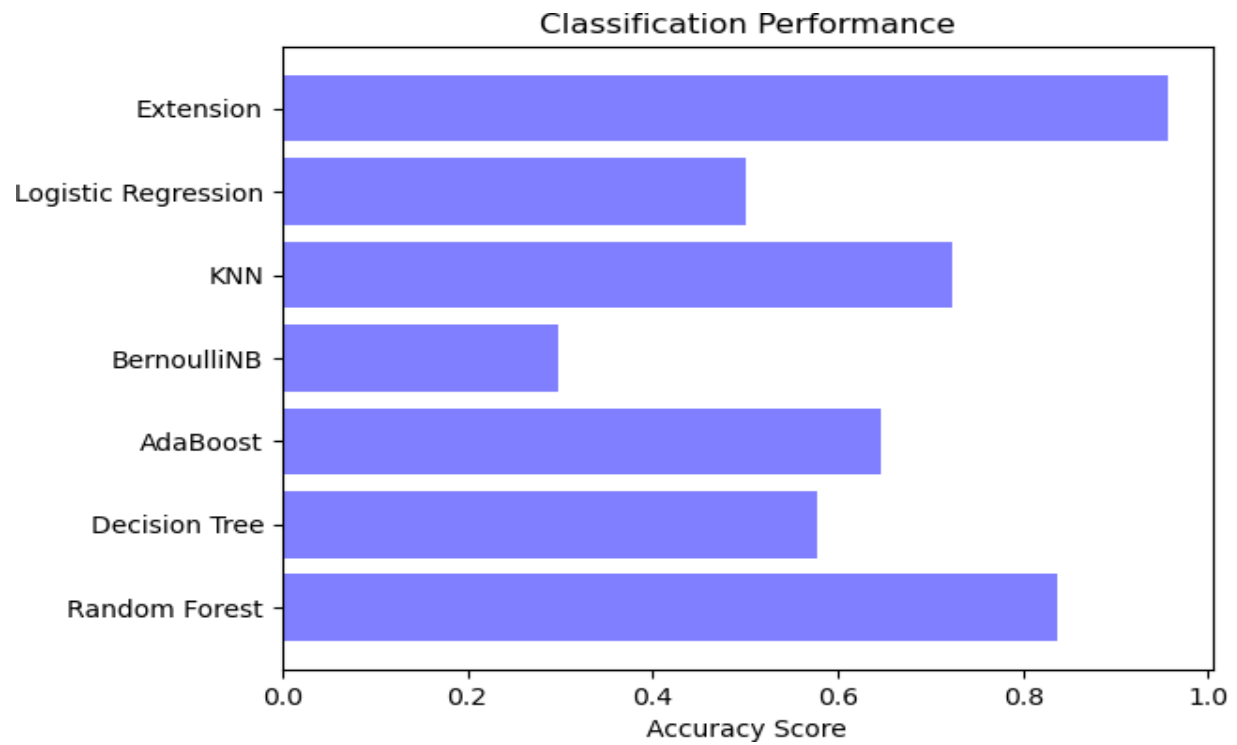


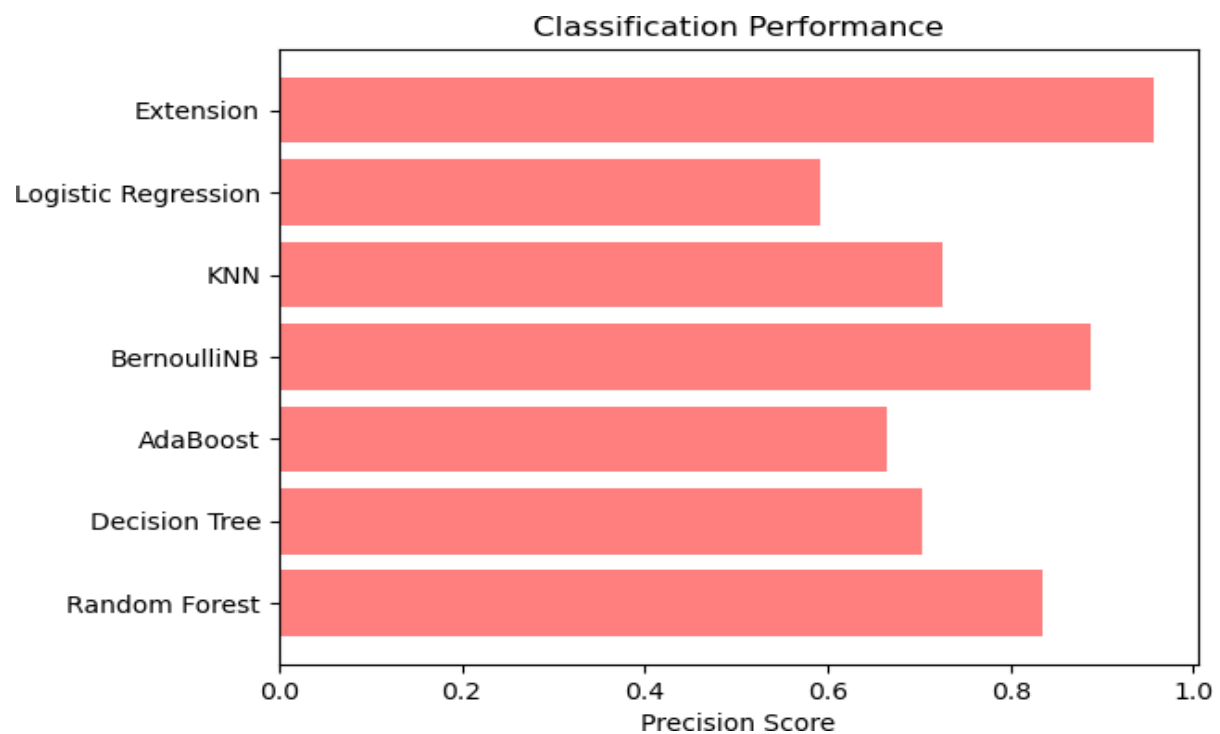
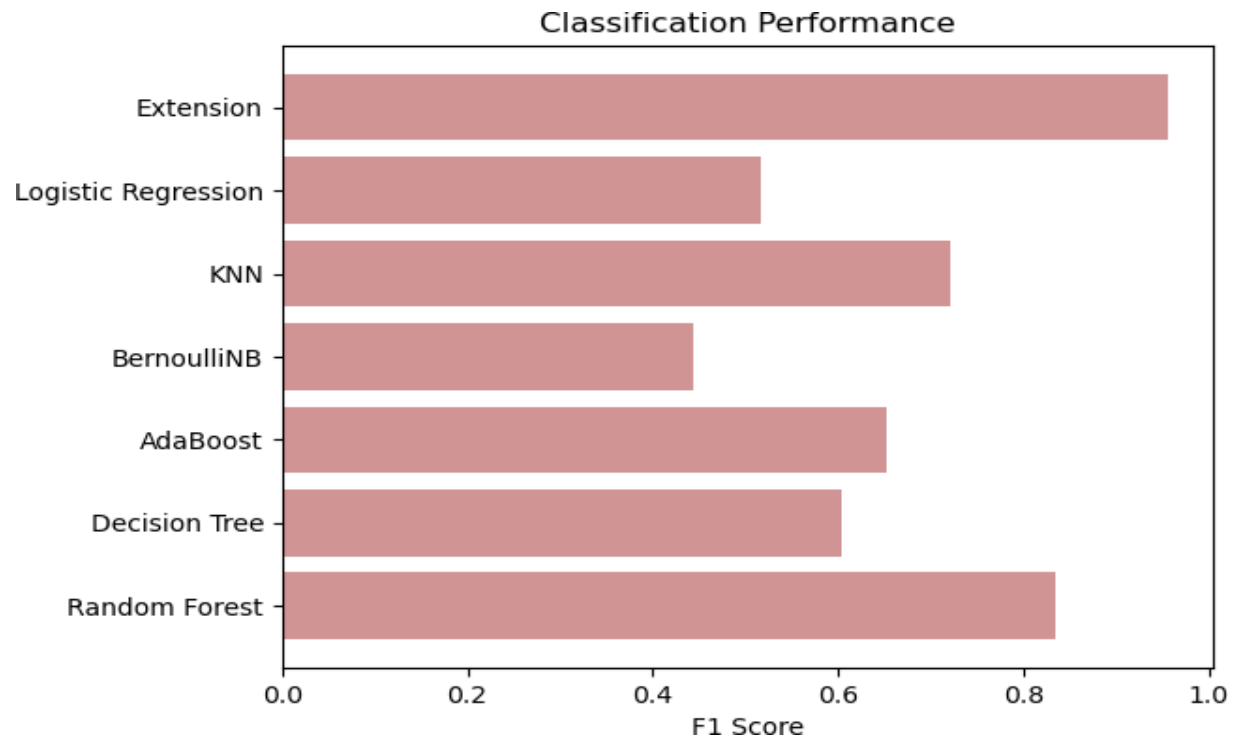
Comparison Graph –Wrapper Based FS



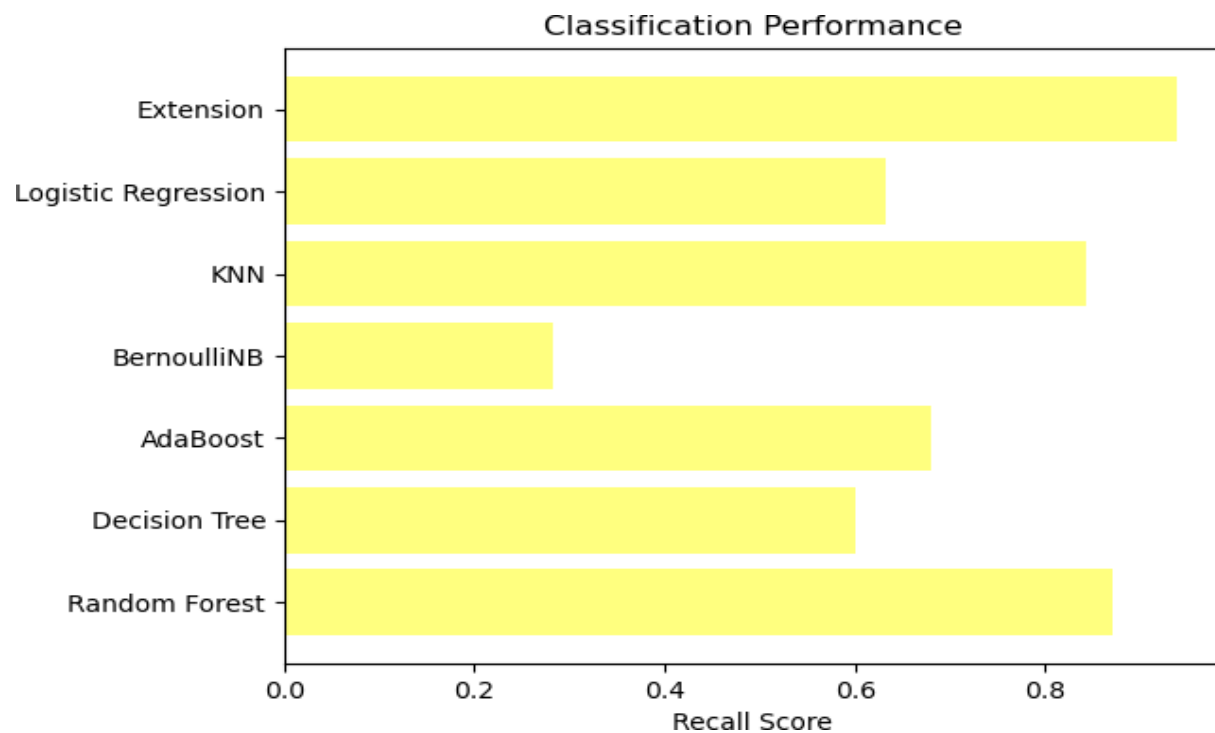
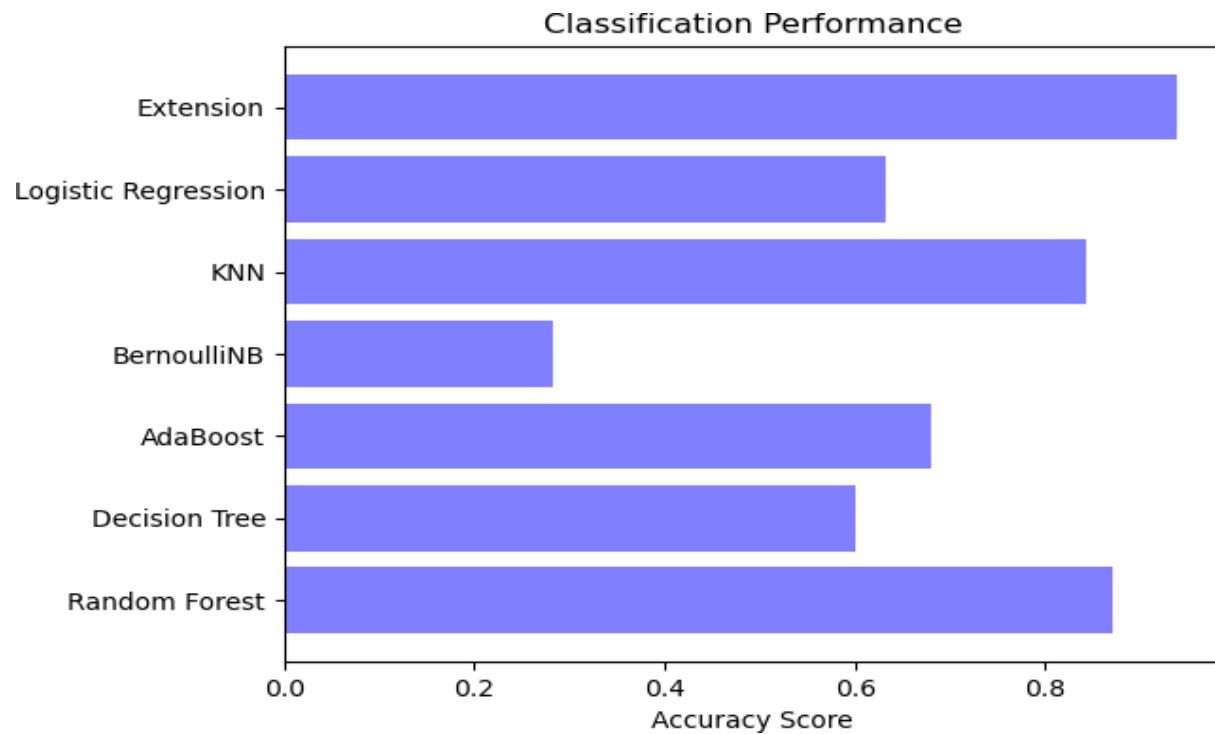


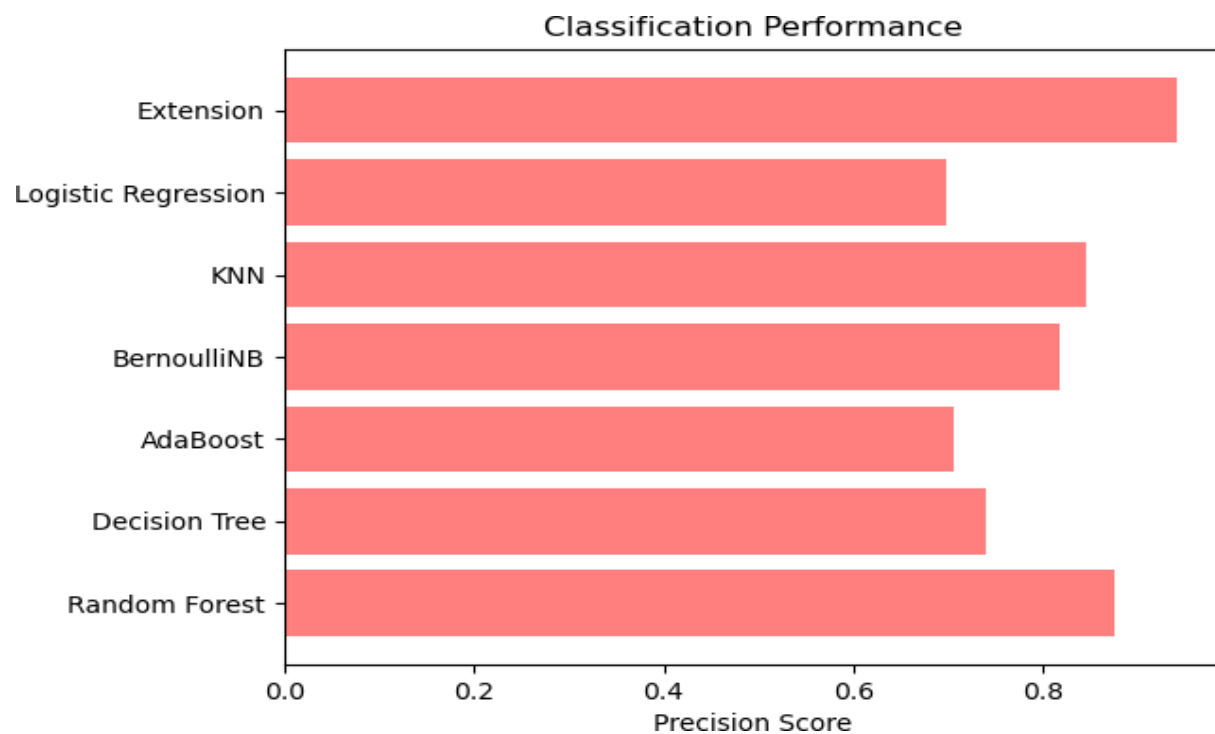
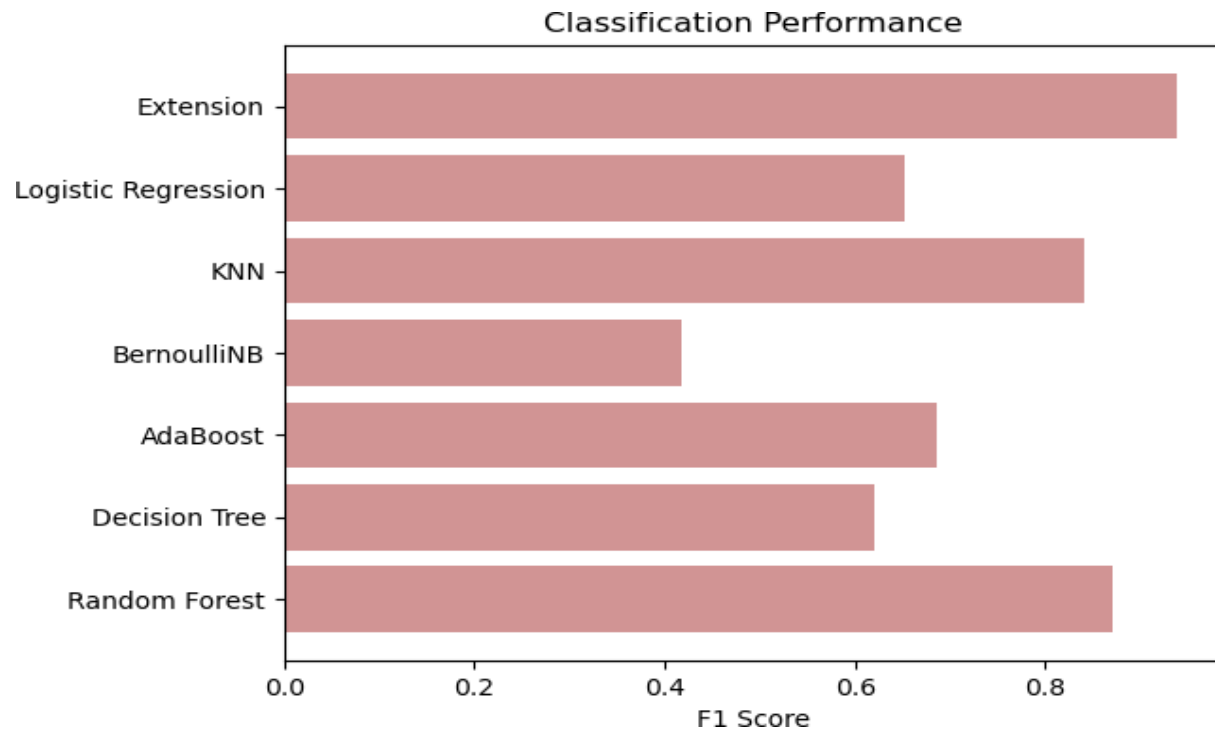
Comparison Graph –Embedded Based FS



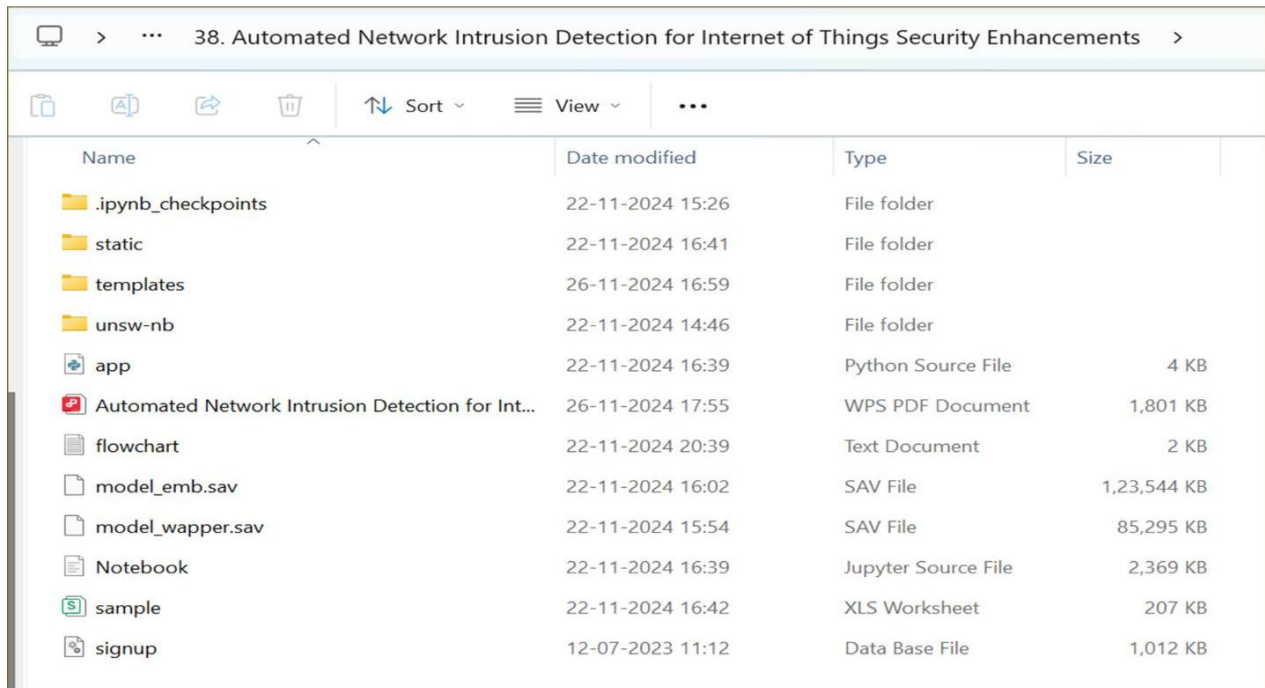


Comparison Graph – PCC Based FS

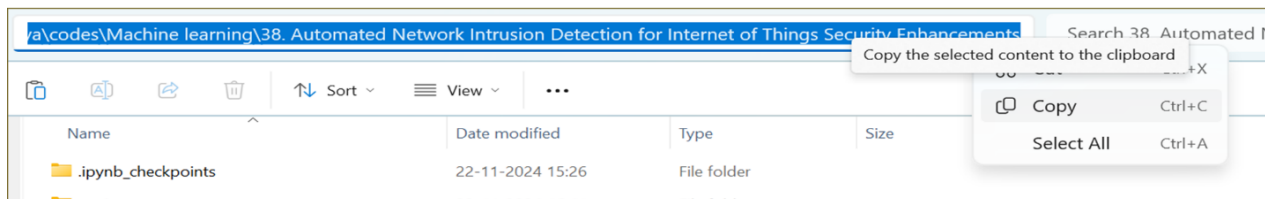




Step - 1



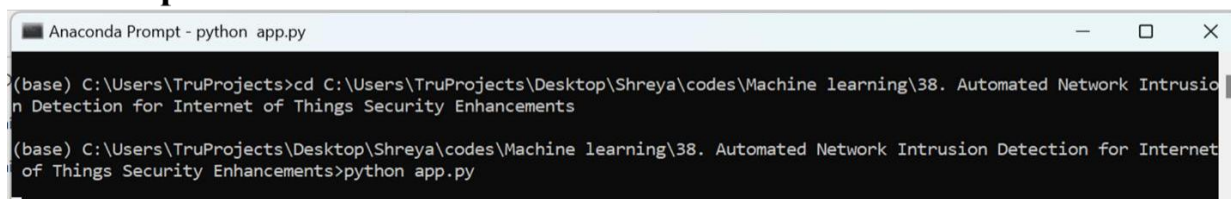
Step - 2



Step - 3



Step - 4



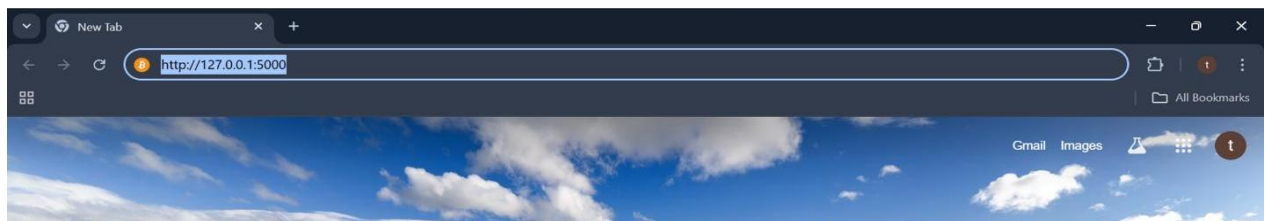
Step -5

```
Select Anaconda Prompt - python app.py

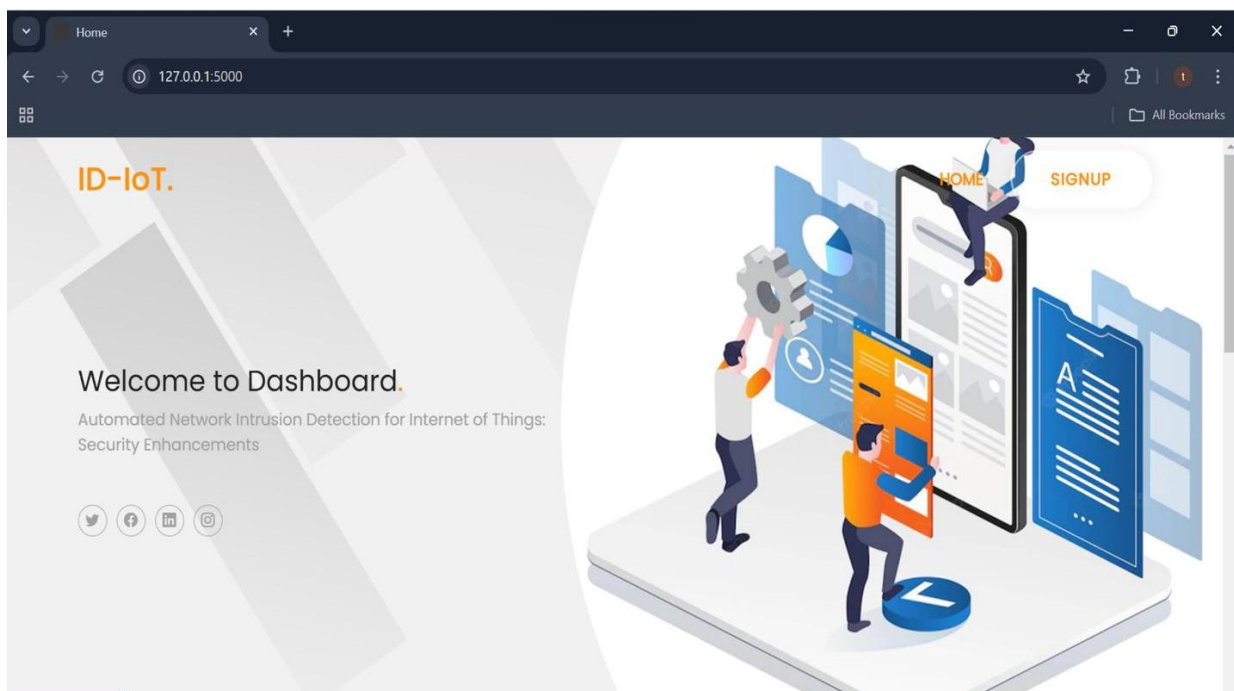
(base) C:\Users\TruProjects>cd C:\Users\TruProjects\Desktop\Shreya\codes\Machine learning\38. Automated Network Intrusion Detection for Internet of Things Security Enhancements

(base) C:\Users\TruProjects\Desktop\Shreya\codes\Machine learning\38. Automated Network Intrusion Detection for Internet of Things Security Enhancements>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
* Debugger is active!
* Debugger PIN: 121-982-402
```

Step - 6



User Interface



Step - 7

New Account

username

Username

name

Name

email

Email

number

Mobile Number

password

Password

☒ Remember me

[Forgot Password](#)

Register

Already have an account? [Sign in](#)

Step - 8

Log In

username
admin

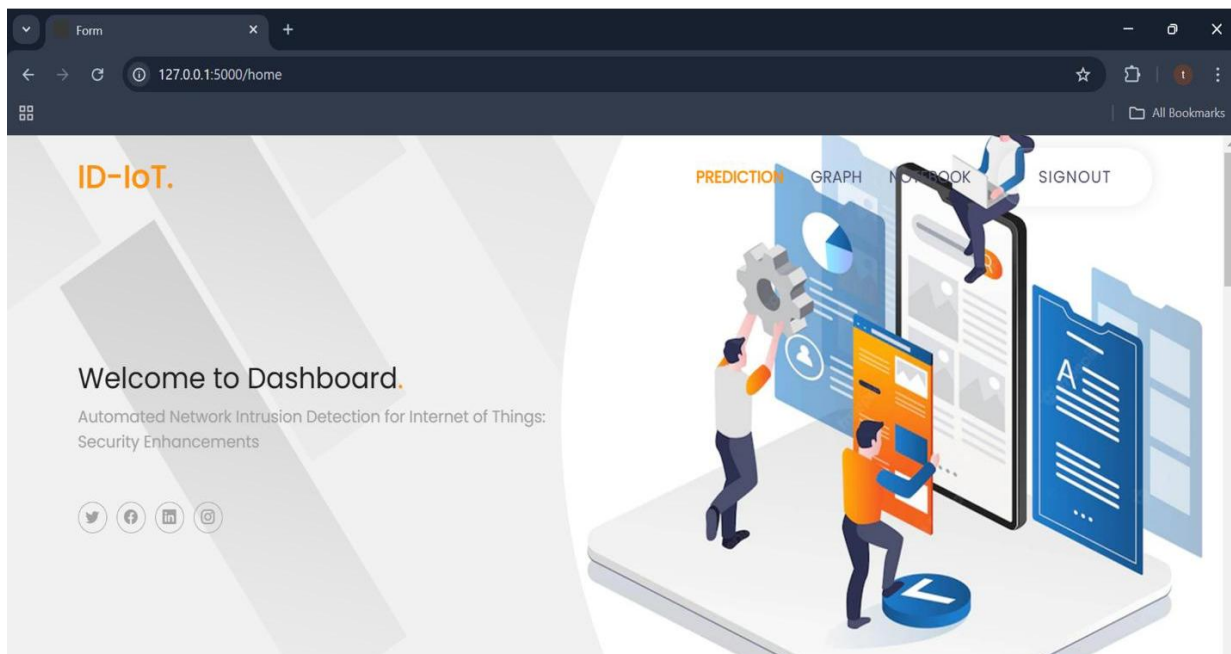
password
.....

☒ Remember me [Forgot Password](#)

Log In

Register here! [Sign Up](#)

Step - 9



Step – 9

Test case 1

RATE: 90909.0902	
STTL: 254	
DLOAD: 0	
ACKDAT: 0	
DMEAN: 0	
CT-SRV-SRC: 20	
CT-DST-SPORT-LTM: 6	
	CT-DST-SRC-LTM 6
	CT-SRC-LTM: 6
	CT-SRV-DST: 18
	Predict

Result:
ATTCAK IS DETECTED, ATTACK TYPE IS DOS!

Step – 9

Test case 2

RATE:	11111.1072
STTL:	254
DLOAD:	0
ACKDAT:	0
DMEAN:	0
CT-SRV-SRC:	7
CT-DST-SPORT-LTM:	2
CT-DST-SRC-LTM:	7
CT-SRC-LTM:	3
CT-SRV-DST:	7
Predict	

Result:

ATTCAK IS DETECTED, ATTACK TYPE IS EXPLOITS!

Step – 9

Test case 3

RATE:	100000
STTL:	254
DLOAD:	0
ACKDAT:	0
DMEAN:	0
CT-SRV-SRC:	14
CT-DST-SRC-LTM:	6
CT-SRV-DST:	14
CT-DST-SPORT-LTM:	6
Predict	

Result:

ATTCaK IS DETECTED, ATTACK TYPE IS FUZZERS!

Step – 9

Test case 4

RATE: 111111.1072	
STTL: 254	
DLOAD: 0	
ACKDAT: 0	
DMEAN: 0	
CT-SRV-SRC: 18	
	CT-DST-SPORT-LTM: 18
	CT-DST-SRC-LTM 18
	CT-SRC-LTM: 20
	CT-SRV-DST: 18
	Predict

Result:

ATTCAK IS DETECTED, ATTACK TYPE IS GENERIC!

Step – 9

Test case 5

RATE: 2042.1751684051578	
STTL: 31	
DLOAD: 5811773.239225941	
ACKDAT: 0.00015101476505793512	
DMEAN: 720	
CT-SRV-SRC: 4	
	CT-DST-SPORT-LTM: 1
	CT-DST-SRC-LTM 1
	CT-SRC-LTM: 5
	CT-SRV-DST: 14
	Predict

Result:**NO ATTACK IS DETECTED, IT IS NORMAL!**

CHAPTER 10: CONCLUSION

In conclusion, the proposed automated Network Intrusion Detection (NID) system for Internet of Things (IoT) environments addresses the pressing challenges of security threats and data integrity. Through the implementation of advanced feature selection techniques, such as Filter and Wrapper Methods alongside an Embedded approach using Lasso with Random Forest and Recursive Feature Elimination (RFE), we enhance the dataset's quality and relevance for training machine learning models. The integration of the Synthetic Minority Over-sampling Technique (SOMTE) effectively mitigates class imbalance, allowing for more accurate detection of intrusions. Among the algorithms evaluated, the Stacking Classifier demonstrates superior performance, combining the strengths of Boosted Decision Trees, Bagging with Random Forest, and LightGBM to achieve a high level of detection accuracy. This robust mechanism not only improves the detection rates but also significantly reduces false alarm rates, addressing critical concerns in IoT security. Ultimately, this system serves as a valuable tool for system administrators, providing an effective solution for safeguarding IoT networks against an ever-evolving landscape of cyber threats while ensuring the integrity and confidentiality of sensitive data.

In future work, we aim to further enhance the automated Network Intrusion Detection (NID) system by exploring advanced techniques such as deep learning algorithms, including convolutional neural networks (CNNs) and recurrent neural networks (RNNs), to improve detection accuracy. Additionally, we will investigate the integration of anomaly detection methods and hybrid models that combine multiple algorithms for better adaptability to diverse attack patterns. Exploring unsupervised learning techniques could also provide valuable insights into emerging threats within IoT environments.

CHAPTER 11. FUTURE SCOPE

1. **Real-time Monitoring and Automatic Threat Response**

In the future, the system can be improved to watch the network live (real-time) and take action on its own when it finds a threat. This helps stop attacks quickly before they cause harm, especially in IoT environments.

2. **Using Advanced Deep Learning Methods**

Techniques like **CNN (Convolutional Neural Networks)**, **LSTM (Long Short-Term Memory)**, and **Transformers** can be used to make the system smarter. These help in better detecting different types of attacks, even new and unknown ones.

3. **Handling Large IoT Networks**

The system can be developed to work well with very large IoT setups (many devices). It should be able to detect threats quickly and efficiently even in complex networks and edge devices (devices near the data source).

4. **Continuous Learning**

The system can learn on its own by updating itself whenever it sees new attack data. This keeps it up to date and improves its ability to detect new types of cyber threats.

5. **Cloud-Based Deployment**

By using cloud technology, the system can work from anywhere. This means easy access, less hardware need, and better monitoring of IoT devices from a central location.

CHAPTER 12. REFERENCES

- [1] O. Abu Alghanam, W. Almobaideen, M. Saadeh, and O. Adwan, “An improved PIO feature selection algorithm for IoT network intrusion detection system based on ensemble learning,” *Exp. Syst. Appl.*, vol. 213, Mar. 2023, Art. no. 118745.
- [2] B. Sharma, L. Sharma, C. Lal, and S. Roy, “Anomaly based network intrusion detection for IoT attacks using deep learning technique,” *Comput. Electr. Eng.*, vol. 107, Apr. 2023, Art. no. 108626.
- [3] I. Debicha, R. Bauwens, T. Debatty, J.-M. Dricot, T. Kenaza, and W. Mees, “TAD: Transfer learning-based multi-adversarial detection of evasion attacks against network intrusion detection systems,” *Future Gener. Comput. Syst.*, vol. 138, pp. 185–197, Jan. 2023.
- [4] W. Wang, S. Jian, Y. Tan, Q. Wu, and C. Huang, “Robust unsupervised network intrusion detection with self-supervised masked context reconstruction,” *Comput. Secur.*, vol. 128, May 2023, Art. no. 103131.
- [5] W. D. Xiong, K. L. Luo, and R. Li, “AIDTF: Adversarial training framework for network intrusion detection,” *Comput. Secur.*, vol. 128, May 2023, Art. no. 103141.
- [6] B. Sharma, L. Sharma, and C. Lal, “Anomaly detection techniques using deep learning in IoT: A survey,” in *Proc. Int. Conf. Comput. Intell. Knowl. Economy (ICCIKE)*, Dec. 2019, pp. 146–149, doi: 10.1109/ICCIKE47802.2019.9004362.
- [7] E. Altulaihan, M. A. Almaiah, and A. Aljughaiman, “Cybersecurity threats, countermeasures and mitigation techniques on the IoT: Future research directions,” *Electronics*, vol. 11, no. 20, p. 3330, Oct. 2022.
- [8] R. A. Nafea and M. Amin Almaiah, “Cyber security threats in cloud: Literature review,” in *Proc. Int. Conf. Inf. Technol. (ICIT)*, Jul. 2021, pp. 779–786.
- [9] D. E. Denning, “Intrusion detection revisited,” *ACM Trans. Inf. Syst. Secur. (TISSEC)*, vol. 5, no. 4, pp. 412–417, 1987.